LLD Phase 5 — PDF 2

Deck of Cards · ATM · Splitwise · Elevator System | Requirements first, hints in the middle, full solutions at the back

> **Colour key. Blue = a note or hint · Red = a common mistake · Green = a rule to remember · Amber = how to say it in an interview.**

Contents

Before you start — what's new in PDF 2 · time boxes · warm-up: two PDF 1 designs from memory, then 12 quick ones

Problem 1 — Deck of Cards: Teen Patti hands (small)

Problem 2 — ATM (core)

Problem 3 — Splitwise (core)

Problem 4 — Elevator System (heavy)

Hints — three levels per problem: a nudge, the shape, the key decision

Solutions 1–4 — clarifying questions · Part A model answer · full code · Part C answers · common mistakes · 10-point checklist

Back pages — warm-up answers · mistake log · re-do schedule

Before you start

The loop is the same as in PDF 1: read once, Part A on paper, one hint level only when stuck for 15 minutes, Part B until the output matches exactly, Part C out loud, then the solution and its checklist. Two things change in this PDF.

What changes	What it means for you
The time box counts now	Stop the timer when your output first matches. Write the time in the tracker, even when it's over. Being slow at first is normal; the re-dos are where the speed comes from.
Patterns come in pairs	State + Chain of Responsibility in the ATM. Strategy + Factory in Splitwise. State + Strategy + Observer + a lock in the elevator. Ask "what varies?" once for *each* part of the problem, not once for the whole problem.
Three new ideas	Comparator as a swappable ranking rule (Deck of Cards). Money as whole paise in a `long` (Splitwise). Time as a tick counter, so a simulation gives the same output every run (Elevator).

Problem	Size	Time box	Format
1. Deck of Cards	small	45 min	10 design + 35 code
2. ATM	core	75 min	20 design + 55 code
3. Splitwise	core	75 min	20 design + 55 code
4. Elevator System	heavy	120 min	30 design + 90 code

> **Why 120 minutes for the elevator?** It's the biggest problem so far: four lift states, two dispatch strategies, listeners, a waiting queue and a concurrency test. Real interview rounds usually give 90–120 minutes for it and expect less than the full solution here. The solutions in this PDF handle every error case and carry comments, so yours can be shorter and still be right.

Warm-up (15 minutes)

Part 1 — from memory. Don't open PDF 1. Write on paper, then check against the back pages of this PDF.

W1	**Vending Machine** (5 min). Draw the state table: three modes (idle, selected, service) × six operations (select, insert, cancel, service on, service off, restock). Fill every cell. Then write the four steps for the coin that completes the price, in order, so that a failed change leaves no trace.
W2	**Parking Lot** (5 min). Which four steps of `park` must be one indivisible step? What must stay **outside** the lock, and why? Name the design for each: the allocation policy chosen by name, the weekend surcharge, the display board, the different rates per vehicle type.

Part 2 — name the pattern (5 min). All from one app: a housing society app in Wakad. Name the pattern, or say "nothing" and what you'd use instead.

1. Maintenance charges are worked out differently for flats, shops and the clubhouse, and the committee changes the formulas every year.
2. Expense approvals: up to Rs 5,000 the treasurer decides, up to Rs 50,000 the secretary, and above that the whole committee.
3. A complaint moves OPEN → ASSIGNED → RESOLVED → CLOSED, or back to OPEN if reopened. What the resident, the plumber and the secretary can each do depends on the status.
4. Flats whose maintenance has been unpaid for 60 days get a reminder.
5. The gate app must use an SMS vendor's SDK, whose method is `sendMsg(String phone, String body, int priority)`. Our code calls `Notifier.visitorArrived(Resident, Visitor)` everywhere.
6. A clubhouse booking has a base fee, plus a sound system, a caterer and decoration, in any combination, each adding to the price.
7. When the water tank drops below 20%: start the pump, alert the secretary, turn the dashboard red. More reactions will be added later.
8. The society is wings, which have floors, which have flats. The committee wants the total area and the total dues at any level.
9. Viewing the CCTV archive needs the secretary's role, and every viewing is logged.
10. A meeting notice has a title and a date (required) and many optional parts: agenda, venue, attachments. Once sent, it never changes.
11. Each parking slot has a number and a wing.
12. Move-in for a new tenant: check documents, issue an access card, register the vehicle, add them to the residents' group. Both the app and the office desk do this.

Problem 1 — Deck of Cards: Teen Patti hands

Small · 45 minutes (10 design + 35 code) · machine-coding format

The story. Every Diwali, the Kulkarni family in Kothrud plays Teen Patti for matchsticks, not money. The arguments about who really won are a family tradition. The younger cousins want an app that deals the cards and settles the showdown.

Functional requirements

R1	A deck has 52 cards: four suits (S spades, H hearts, D diamonds, C clubs) × 13 ranks (2, 3, …, 10, J, Q, K, A). A card is written rank then suit: `10H`, `AS`, `7D`. A new deck is in this exact order: all the spades from 2 up to A, then the hearts, then the diamonds, then the clubs. The top of the deck is the first card. The program starts with a new deck.
R2	Shuffling takes a seed, so a test can repeat it. Shuffle exactly like this, so your output matches: `Collections.shuffle(cards, new Random(seed))`.
R3	Dealing gives 3 cards to each named player from the top of the deck, one card at a time, going round the players three times. Refused, before anything is dealt, if a named player already has a hand this round (or is named twice), or if there aren't enough cards.
R4	For testing, a player can be given 3 named cards, taken out of the deck. Refused, checked in this order, if: the player already has a hand, it isn't exactly 3 cards, a card is written wrongly, a card appears twice, or a card isn't in the deck. A refused command takes nothing from the deck.
R5	Hand types, best first — see the table below. "In a row" means consecutive ranks in the order 2, 3, …, 10, J, Q, K, A. The ace is also low, but only in A-2-3: that's a sequence, and the **lowest** one. Q-K-A is the highest. K-A-2 is not a sequence.
R6	Two hands of the same type are compared like this, and suits never matter. **Trail:** the higher rank wins. **Sequences:** the higher top card wins (A-2-3 counts as 3-high). **Color and high card:** compare the highest cards, then the middle ones, then the lowest. **Pair:** the higher pair wins; if the pairs are equal, the third card decides. If everything is equal, the hands tie.
R7	The showdown ranks every hand at the table, best first. Tied hands keep the order in which the players joined the table. It needs at least 2 hands, and it ends the round: the table is empty afterwards. (The cards don't go back into the deck.)
R8	The variant is STANDARD (the default) or MUFLIS, where the ranking is exactly reversed: the lowest hand wins.

Type (best first)	Example	Rule
trail	`7H 7S 7D`	three cards of one rank
pure sequence	`4S 5S 6S`	three ranks in a row, all one suit
sequence	`4S 5H 6D`	three ranks in a row, mixed suits
color	`2H 9H KH`	all one suit, not in a row
pair	`9S 9D 4C`	two cards of one rank
high card	`2C 10S QS`	anything else

Non-functional requirements

NF1	A new variant (another ranking rule) must not change the card, deck or hand code.
NF2	A wrong card must be impossible inside the program: no rank 15, no suit 'X'. Text is checked once, when it comes in.

Out of scope: betting, blind and seen players, jokers, several tables at once.

Commands and exact output

Command	Output
`NEW_DECK`	`New deck: 52 cards` (also empties the table)
`SHUFFLE seed`	`Shuffled with seed 2026`
`DEAL name name …`	one line per player, in the order named: `Asha: 2S 4S 6S (color)`, then `Cards left: 46`
`HAND name c1 c2 c3`	`Asha: AS 2D 3C (sequence)` (cards in the order given; type names in lower case, as in the table above)
`SHOW`	one line per hand, best first, in the same format; then `Winner: Ravi`, or `Tie: Asha, Ravi` when several hands share the top
`VARIANT name`	`Variant: MUFLIS`

Error	Output
`a player already has a hand (DEAL or HAND)`	`Asha already has a hand`
`not enough cards to deal`	`Not enough cards: need 27, have 25`
`not exactly 3 cards`	`A hand needs exactly 3 cards`
`a badly written card`	`Invalid card 1X`
`the same card twice`	`Duplicate card 9C`
`a card that isn't in the deck`	`4H is not in the deck`
`SHOW with fewer than 2 hands`	`Need at least 2 hands to show`
`unknown variant`	`Unknown variant JOKER`

Which error wins? For HAND, the order in R4. For DEAL, the players are checked (in the order named) before the number of cards.

Part A — design

1	For each, say enum, record or class, and why: suit, rank, card, deck, hand, hand type, the ranking rule.
2	Where is "what type of hand is this?" worked out, and when? Think about who holds the cards, and whether a hand can ever change.
3	The ranking: `Comparable` (inside the hand) or a `Comparator` (outside it)? Let R8 and NF1 decide.
4	How do you make shuffling testable? What would you do if no seed were given?

Part B — code

input.txt

```
DEAL Asha Ravi
SHOW
SHUFFLE 2026
DEAL Asha Ravi Meera
DEAL Kiran Asha
SHOW
NEW_DECK
HAND Asha AS 2D 3C
HAND Ravi QH KD AC
HAND Meera 10S 10D 4H
HAND Kiran 9C 9H
HAND Kiran 9C 1X 4D
HAND Kiran 9C 9C 4D
HAND Kiran 9C 4H 5D
HAND Meera 9C 5S 6D
SHOW
SHOW
HAND Meera 4C 8H KH
HAND Asha 5H 8C KS
HAND Ravi 5S 8D KC
SHOW
VARIANT MUFLIS
HAND Meera 7C 7D 7H
HAND Asha 2S 3D 5C
HAND Ravi 2C 6C 9C
SHOW
VARIANT JOKER
DEAL P1 P2 P3 P4 P5 P6 P7 P8 P9
```

expected output

```
Asha: 2S 4S 6S (color)
Ravi: 3S 5S 7S (color)
Cards left: 46
Ravi: 3S 5S 7S (color)
Asha: 2S 4S 6S (color)
Winner: Ravi
Shuffled with seed 2026
Asha: 8D 2D 2H (pair)
Ravi: 2C 10S QS (high card)
Meera: KD AD 4C (high card)
Cards left: 37
Asha already has a hand
Asha: 8D 2D 2H (pair)
Meera: KD AD 4C (high card)
Ravi: 2C 10S QS (high card)
Winner: Asha
New deck: 52 cards
Asha: AS 2D 3C (sequence)
Ravi: QH KD AC (sequence)
Meera: 10S 10D 4H (pair)
A hand needs exactly 3 cards
Invalid card 1X
Duplicate card 9C
4H is not in the deck
Meera already has a hand
Ravi: QH KD AC (sequence)
Asha: AS 2D 3C (sequence)
Meera: 10S 10D 4H (pair)
Winner: Ravi
Need at least 2 hands to show
Meera: 4C 8H KH (high card)
Asha: 5H 8C KS (high card)
Ravi: 5S 8D KC (high card)
Asha: 5H 8C KS (high card)
Ravi: 5S 8D KC (high card)
Meera: 4C 8H KH (high card)
Tie: Asha, Ravi
Variant: MUFLIS
Meera: 7C 7D 7H (trail)
Asha: 2S 3D 5C (high card)
Ravi: 2C 6C 9C (color)
Asha: 2S 3D 5C (high card)
Ravi: 2C 6C 9C (color)
Meera: 7C 7D 7H (trail)
Winner: Asha
Unknown variant JOKER
Not enough cards: need 27, have 25
```

> **Look at the first DEAL.** An unshuffled deck deals Asha 2S 4S 6S and Ravi 3S 5S 7S: two colors. That's why decks get shuffled. If your shuffled hands differ from the expected ones, check the order in which you build the deck (R1) before anything else.

Part C — pressure questions

1	**Change.** "Most of our relatives play A-2-3 as the second-best sequence, just below Q-K-A." What changes in your code, and what doesn't?
2	**Change.** "For the big Diwali party we shuffle two decks together." Which of your rules and checks break? Look closely at R4, and at how you spot duplicates.
3	**Shuffling.** A cousin writes his own shuffle: for each position i, swap card i with a random card from anywhere in the deck (0 to 51). Is that fair? What does `Collections.shuffle` do instead?
4	**Testing.** "How do you unit-test SHOW when dealing is random?"
5	**Defend it.** "Enums and records are overkill. A card is an int from 0 to 51: suit = n / 13, rank = n % 13."

Problem 2 — ATM

Core · 75 minutes (20 design + 55 code) · machine-coding format

The story. Sahyadri Co-operative Bank has an ATM outside its Deccan Gymkhana branch. It holds Rs 500, Rs 200 and Rs 100 notes in three cassettes. The bank has two rules it never breaks: a customer must never lose money because of the machine, and the bank must never pay out money it didn't debit.

Functional requirements

R1	The bank keeps accounts (an ID and a balance in whole rupees) and cards. Each card belongs to one account and has a PIN. Two cards can share one account (a joint account).
R2	A session: insert a card, enter the PIN, then check the balance and withdraw as many times as you like, then eject. One card at a time.
R3	A wrong PIN can be tried again. Three wrong PINs in a row **block** the card, end the session and give the card back. A blocked card is refused from then on. The count of wrong PINs belongs to the card: ejecting and inserting it again doesn't reset the count. Only a correct PIN does.
R4	A withdrawal must be a positive multiple of Rs 100, and at most Rs 10,000. Checked in this order: the amount, the limit, whether the machine's notes can make the amount, then the account balance.
R5	Notes are chosen largest first: the Rs 500 cassette gives as many notes as it can without going over the amount, then the Rs 200 cassette does the same with what's left, then the Rs 100. If anything is left at the end, the withdrawal is refused.
R6	A withdrawal happens completely or not at all. If it's refused for any reason, the balance and the notes in the machine stay exactly as they were.
R7	The operator loads notes only when no card is inside. The machine starts with no notes. A cash report shows each cassette, largest note first, and the total.

Non-functional requirements

NF1	Other ATMs have other cassettes (no Rs 200, or an extra Rs 2,000). Changing which cassettes a machine has must not change the dispensing code.
NF2	New session steps or modes (a deposit, a maintenance mode) should be easy to add.
NF3	Many ATMs talk to the same bank. Two withdrawals from one account at the same moment must never take its balance below zero.

Out of scope: deposits, transfers, PIN changes, daily limits, receipts, network failures (Part C asks about one).

Commands and exact output

Command	Output
`ADD_ACCOUNT id balance`	`Account S1 opened with Rs 5000`
`ADD_CARD number account pin`	`Card 1111 issued for S1`
`LOAD note count`	`Loaded Rs 500 x 10`
`CASH`	`Cash: 500x10 200x4 100x0, total Rs 5800`
`INSERT card`	`Card 1111 inserted, enter PIN`
`PIN pin`	`PIN accepted`
`BALANCE`	`Balance: Rs 5000`
`WITHDRAW amount`	`Dispensed Rs 1700 (500x3 200x1), balance Rs 3300` (only the notes used, largest first)
`EJECT`	`Card 1111 ejected`

Error	Output
`unknown account / unknown card`	`Unknown account S9` / `Unknown card 9999`
`inserting a blocked card`	`Card 3333 is blocked`
`PIN, BALANCE, WITHDRAW or EJECT with no card`	`Insert a card first`
`INSERT while a card is inside`	`A card is already inserted`
`BALANCE or WITHDRAW before the PIN`	`Enter your PIN first`
`PIN after it was accepted`	`PIN already accepted`
`a wrong PIN`	`Wrong PIN, 2 attempts left` / `Wrong PIN, 1 attempt left`
`the third wrong PIN in a row`	`Wrong PIN, card 3333 blocked` (the session ends)
`a bad amount`	`Amount must be a positive multiple of Rs 100`
`over the limit`	`Maximum Rs 10000 per withdrawal`
`the notes can't make the amount`	`Cannot dispense Rs 600 with the notes available`
`not enough money in the account`	`Insufficient balance`
`LOAD while a card is inside`	`Cannot load cash during a session`
`a note with no cassette`	`Rs 50 notes are not supported`
`an ID used twice (not in the test)`	`Account S1 already exists` / `Card 1111 already exists`

Which error wins? The session comes first: with no card inside, every customer command gives "Insert a card first", and INSERT with a card inside always gives "A card is already inserted". For INSERT, "Unknown card" comes before "is blocked". For LOAD, "Cannot load cash during a session" comes before the note check. For WITHDRAW, the order in R4.

Part A — design

1	Draw the state table: one row per state, one column per operation (insert, PIN, balance, withdraw, eject, load). Fill every cell.
2	Where does the wrong-PIN count live: in the ATM, in the session, or on the card? What goes wrong with each wrong choice?
3	Draw the chain of cassettes. What does one link do with an amount? How do you find out whether an amount can be paid **without** changing any counts?
4	Write the steps of a withdrawal in order. Mark the **commit point**: the step after which "just throw an error" is no longer enough.

Part B — code

input.txt

```
ADD_ACCOUNT S1 5000
ADD_ACCOUNT S2 800
ADD_CARD 1111 S1 4321
ADD_CARD 2222 S1 1234
ADD_CARD 3333 S2 2580
ADD_CARD 4444 S9 1111
BALANCE
LOAD 500 10
LOAD 200 4
LOAD 50 10
CASH
INSERT 9999
INSERT 1111
INSERT 2222
BALANCE
LOAD 100 10
PIN 1234
PIN 4321
PIN 4321
BALANCE
WITHDRAW 250
WITHDRAW 12000
WITHDRAW 1700
WITHDRAW 600
WITHDRAW 3500
EJECT
EJECT
INSERT 1111
PIN 0000
EJECT
INSERT 3333
PIN 0000
EJECT
INSERT 3333
PIN 1111
PIN 9999
WITHDRAW 100
INSERT 3333
INSERT 2222
PIN 1234
BALANCE
WITHDRAW 3000
WITHDRAW 600
EJECT
LOAD 100 10
CASH
```

expected output

```
Account S1 opened with Rs 5000
Account S2 opened with Rs 800
Card 1111 issued for S1
Card 2222 issued for S1
Card 3333 issued for S2
Unknown account S9
Insert a card first
Loaded Rs 500 x 10
Loaded Rs 200 x 4
Rs 50 notes are not supported
Cash: 500x10 200x4 100x0, total Rs 5800
Unknown card 9999
Card 1111 inserted, enter PIN
A card is already inserted
Enter your PIN first
Cannot load cash during a session
Wrong PIN, 2 attempts left
PIN accepted
PIN already accepted
Balance: Rs 5000
Amount must be a positive multiple of Rs 100
Maximum Rs 10000 per withdrawal
Dispensed Rs 1700 (500x3 200x1), balance Rs 3300
Cannot dispense Rs 600 with the notes available
Insufficient balance
Card 1111 ejected
Insert a card first
Card 1111 inserted, enter PIN
Wrong PIN, 2 attempts left
Card 1111 ejected
Card 3333 inserted, enter PIN
Wrong PIN, 2 attempts left
Card 3333 ejected
Card 3333 inserted, enter PIN
Wrong PIN, 1 attempt left
Wrong PIN, card 3333 blocked
Insert a card first
Card 3333 is blocked
Card 2222 inserted, enter PIN
PIN accepted
Balance: Rs 3300
Dispensed Rs 3000 (500x6), balance Rs 300
Cannot dispense Rs 600 with the notes available
Card 2222 ejected
Loaded Rs 100 x 10
Cash: 500x1 200x3 100x10, total Rs 2100
```

Two lines worth a second look. `WITHDRAW 600` fails even though three Rs 200 notes are left: largest-first takes the Rs 500 note and then can't pay the last Rs 100. That's R5 working as written (Part C 2 asks you to improve it). And card 1111's wrong PIN after a good one says "2 attempts left", not "1": the correct PIN reset the count.

Part C — pressure questions

1	**Change.** "Add cash deposits: the customer feeds in notes, the machine counts them, puts them in its cassettes and credits the account." What changes? Count the classes you'd touch.
2	**Change.** "WITHDRAW 600 failed with one Rs 500 and three Rs 200 notes left, though 200 + 200 + 200 works. Fix it." Does the chain of cassettes still fit?
3	**Failure.** "The bank has debited the account, and then the dispenser jams." What must happen, and in what order? What must the system remember?
4	**Concurrency.** "Husband and wife withdraw Rs 4,000 each from their joint account at two ATMs at the same moment. The balance is Rs 5,000." What protects them in your code? What protects them when the bank runs three servers?
5	**Defend it.** "A chain of cassettes for three notes? A `for` loop over [500, 200, 100] is simpler."

Problem 3 — Splitwise

Core · 75 minutes (20 design + 55 code) · machine-coding format

The story. Four friends share a flat in Baner. They split the rent, the groceries and the Wi-Fi bill, and now and then a weekend trip to Lonavala. They want to know where each of them stands, and the fewest payments that would settle everything.

Functional requirements

R1	Users are added by name (one word). Names are unique.
R2	An expense has a payer, an amount, a split type, and the people it's split between. The payer may or may not be one of those people.
R3	Amounts are in rupees with up to 2 decimal places (`1200` , `450.50`). Store and calculate them in **paise**, as whole numbers. Print them with exactly 2 decimals (`1200.00`).
R4	Split types. **EQUAL:** everyone gets the same share; the entries are names (`Asha Ravi Meera`). **EXACT:** each share is given (`Asha:500 Ravi:500`), and the shares must add up to the amount exactly. **PERCENT:** each percentage is given as a whole number (`Asha:33 Ravi:33 Meera:34`); they must add up to 100, and a share is amount × percent / 100.
R5	Rounding, for EQUAL and PERCENT: round every share **down** to the paisa, then give the leftover paise, one each, to the people in the order listed. So Rs 1000 split equally three ways is 333.34, 333.33 and 333.33.
R6	An expense is checked completely before it changes anything. The order: the payer exists, the amount is valid, the split type exists, there's at least one person, then each person in the order listed (unknown, then listed twice), then the split type's own rules (the form of each entry, then the total).
R7	A balance is one number per person: everything they've paid minus everything they owe, whoever it was for. Balances are shown in the order people were added.
R8	A settlement records that one person paid another directly: the payer's balance goes up by the amount and the receiver's goes down. Any positive amount is allowed.
R9	Simplify suggests payments that bring everyone to zero. Repeat: the person who owes the most pays the person who is owed the most, as much as one of them needs. A tie goes to whoever was added first. Simplify only suggests; it changes nothing.

Non-functional requirements

NF1	A new split type (for example SHARES: `Asha:2 Ravi:1`) must not change the existing split code, the balances or the driver.
NF2	Money is never a `double` .
NF3	Two flatmates adding expenses from their phones at the same moment must not corrupt the balances.

Out of scope: groups, other currencies, editing or deleting expenses (Part C asks about it), notifications.

Commands and exact output

Command	Output
`ADD_USER name`	`Added Asha`
`EXPENSE payer amount TYPE entries …`	`Ravi paid Rs 1000.00, split EQUAL: Asha 333.34, Ravi 333.33, Meera 333.33` (the shares, in the order listed)
`SETTLE from to amount`	`Ravi paid Asha Rs 200.00`
`BALANCES`	one line per user: `Meera gets back Rs 801.59` / `Ravi owes Rs 688.41` / `Kiran is settled up`
`SIMPLIFY`	one line per payment: `Ravi pays Meera Rs 688.41` ; or `Everyone is settled up`

Error	Output
`a name that's taken`	`User Asha already exists`
`an unknown user`	`Unknown user Nikhil`
`a bad amount (not a number, not above zero, more than 2 decimals), also inside an EXACT entry`	`Invalid amount 12.345`
`an unknown split type`	`Unknown split type HALF`
`no entries at all`	`Nobody to split with`
`the same person twice`	`Ravi is listed twice`
`an entry in the wrong form for its type (a value in EQUAL, a missing value, a percentage that isn't a whole number)`	`Bad entry Ravi:250 for EQUAL`
`EXACT shares with the wrong total`	`Exact amounts add up to Rs 900.00, not Rs 1000.00`
`percentages with the wrong total`	`Percentages add up to 90, not 100`
`settling with yourself`	`Cannot settle with yourself`

Which error wins? For EXPENSE, the order in R6. For SETTLE: both users exist (the payer first), then "yourself", then the amount.

Part A — design

1	List the entities. Which ones are records? Where does "how to split" live?
2	NF1: show how SHARES would be added. Which files change?
3	Balances: one net number per person, or a who-owes-whom table for every pair? Give one advantage of each.
4	R5: how do you split Rs 100 three ways so the shares add up exactly? And why not `double` ?

Part B — code

input.txt

```
ADD_USER Asha
ADD_USER Ravi
ADD_USER Meera
ADD_USER Kiran
ADD_USER Asha
EXPENSE Asha 1200 EQUAL Asha Ravi Meera Kiran
EXPENSE Ravi 1000 EQUAL Asha Ravi Meera
EXPENSE Meera 2000 EXACT Asha:500 Ravi:500 Kiran:1000
EXPENSE Kiran 999.50 PERCENT Asha:33 Ravi:33 Meera:34
EXPENSE Asha 450.50 EQUAL Ravi Meera
EXPENSE Nikhil 500 EQUAL Asha Ravi
EXPENSE Asha 12.345 EQUAL Ravi
EXPENSE Asha 500 HALF Ravi Meera
EXPENSE Asha 500 EQUAL Ravi Nikhil
EXPENSE Asha 500 EQUAL Ravi Meera Ravi
EXPENSE Asha 500 EQUAL Ravi:250 Meera
EXPENSE Asha 500 EQUAL
EXPENSE Asha 1000 EXACT Ravi:300 Meera:600
EXPENSE Asha 1000 PERCENT Ravi:50 Meera:40
BALANCES
SIMPLIFY
SETTLE Ravi Asha 200
SETTLE Ravi Ravi 10
BALANCES
SIMPLIFY
SETTLE Ravi Meera 488.41
SETTLE Kiran Meera 300.50
SETTLE Asha Meera 12.68
BALANCES
SIMPLIFY
```

expected output

```
Added Asha
Added Ravi
Added Meera
Added Kiran
User Asha already exists
Asha paid Rs 1200.00, split EQUAL: Asha 300.00, Ravi 300.00, Meera 300.00, Kiran 300.00
Ravi paid Rs 1000.00, split EQUAL: Asha 333.34, Ravi 333.33, Meera 333.33
Meera paid Rs 2000.00, split EXACT: Asha 500.00, Ravi 500.00, Kiran 1000.00
Kiran paid Rs 999.50, split PERCENT: Asha 329.84, Ravi 329.83, Meera 339.83
Asha paid Rs 450.50, split EQUAL: Ravi 225.25, Meera 225.25
Unknown user Nikhil
Invalid amount 12.345
Unknown split type HALF
Unknown user Nikhil
Ravi is listed twice
Bad entry Ravi:250 for EQUAL
Nobody to split with
Exact amounts add up to Rs 900.00, not Rs 1000.00
Percentages add up to 90, not 100
Asha gets back Rs 187.32
Ravi owes Rs 688.41
Meera gets back Rs 801.59
Kiran owes Rs 300.50
Ravi pays Meera Rs 688.41
Kiran pays Asha Rs 187.32
Kiran pays Meera Rs 113.18
Ravi paid Asha Rs 200.00
Cannot settle with yourself
Asha owes Rs 12.68
Ravi owes Rs 488.41
Meera gets back Rs 801.59
Kiran owes Rs 300.50
Ravi pays Meera Rs 488.41
Kiran pays Meera Rs 300.50
Asha pays Meera Rs 12.68
Ravi paid Meera Rs 488.41
Kiran paid Meera Rs 300.50
Asha paid Meera Rs 12.68
Asha is settled up
Ravi is settled up
Meera is settled up
Kiran is settled up
Everyone is settled up
```

> **Two things to notice.** Kiran's Rs 999.50 at 33/33/34 leaves one paisa over after rounding down, and it goes to Asha, the first listed (329.84). And after `SETTLE Ravi Asha 200`, Asha *owes* Rs 12.68: she was owed only Rs 187.32 in total, and Ravi paid her Rs 200. Net balances don't care who paid whom; they only add up.

Part C — pressure questions

1	**Change.** "Add SHARES: `Asha:2 Ravi:1 Meera:1` means Asha pays half and the others a quarter each." Write it.
2	**Change.** "Let people delete an expense they added by mistake." What must you start storing? Should balances still be stored at all?
3	**Concurrency.** "Two flatmates add an expense at the same moment." What can go wrong when an expense is recorded, and why isn't a `ConcurrentHashMap` enough on its own? And with three servers?
4	**Algorithm.** "Is SIMPLIFY always the fewest payments?" Try these balances: Asha +400, Ravi +300, Meera +300, Kiran −600, Neha −400.
5	**Defend it.** "Why a class per split type? A `switch` inside `addExpense` is twenty lines."

Problem 4 — Elevator System

Heavy · 120 minutes (30 design + 90 code) · machine-coding format, with a concurrency test

The story. Tower B of an IT park in Hinjewadi has ten floors (0 to 9) and three lifts. At 9:30 every morning, everyone arrives at once. The facilities team wants a lift controller they can tune: try different ways of choosing a lift, take a lift out for maintenance, and never send two lifts for one button.

Time moves in **ticks**. A lift moves one floor per tick, and its doors stay open for one tick. The STEP command runs ticks, so every run gives the same output.

Functional requirements

R1	The building has F floors, numbered from 0 (the ground floor), and L lifts named L1, L2, ... Every lift starts idle at floor 0 with its doors closed.
R2	**Hall call:** someone on a floor presses UP or DOWN. The top floor has no UP button and the ground floor has no DOWN button. The controller chooses a lift with the current dispatch strategy, and that floor becomes one of the lift's stops.
R3	The pressed button stays **lit** until the chosen lift stops at that floor. Pressing a lit button again changes nothing. If a lift is standing at that floor with its doors open, no lift is needed: it's "already here".
R4	**Car call:** someone inside a lift presses a floor, and it becomes one of that lift's stops. If that lift's doors are open at that floor already, nothing happens.
R5	A lift stops at every floor in its stop list when it reaches it, whichever button the stop came from. It keeps going in its direction while it has stops ahead, and only then turns around.
R6	Each tick, the lifts act one after another (L1 first). What a lift does depends on its state at the start of the tick, as in the table below. When a lift opens its doors, it announces it: `t=3: L1 stops at 3`.
R7	**Dispatch strategies.** Only lifts not in maintenance are considered, and a tie goes to the lower lift number. **NEAREST** (the default): the smallest distance between the lift's floor and the call's floor. **LEAST_TRAVEL**: an estimate of how far the lift travels before it reaches the call, if it carries on in its current direction. A lift with no direction: the distance. A call **strictly ahead** in its direction: the distance. Otherwise: from the lift to its farthest stop in that direction (its own floor if it has none there), plus from there back to the call. The operator can switch strategies at any time.
R8	**Maintenance:** a lift can go into maintenance only when it's idle with no stops. A lift in maintenance takes no stops. When it comes back, it's idle at the same floor.
R9	If a hall call comes while every lift is in maintenance, it **waits** in a queue (pressing it again changes nothing). As soon as a lift is back in service, the waiting calls are dispatched in the order they came.
R10	Status shows the time, then each lift (its floor, its state, and its stops in floor order), then the waiting calls, if there are any.

State at the start of the tick	What the lift does this tick
IDLE	No stops: nothing. A stop at its own floor: remove it and open the doors. Otherwise: turn toward the nearest stop (a tie goes up), become MOVING, and move one floor in this same tick (opening the doors if that floor is a stop).
MOVING	Move one floor. If the new floor is a stop: remove it and open the doors.
DOORS OPEN	The doors close; the lift doesn't move this tick. No stops left: IDLE, with no direction. Otherwise: keep its direction if there's a stop ahead, else turn around; then MOVING. (A lift that opened its doors while idle has no direction yet: it turns toward the nearest stop.)
MAINTENANCE	Nothing.

> **A worked example.** L1 is idle at floor 0, and someone inside presses 3 (`GO L1 3`). **t=1:** idle with a stop, so it heads up and moves to 1. **t=2:** moves to 2. **t=3:** moves to 3, which is a stop: the doors open and it prints `t=3: L1 stops at 3`. **t=4:** the doors close; no stops are left, so it's idle again.
> **LEAST_TRAVEL, by example:** L1 is at floor 4, moving up, with a stop at 6. A call comes from floor 2. NEAREST says 2 floors away. LEAST_TRAVEL says $(6 - 4) + (6 - 2) = 6$, because L1 goes up to 6 before it can come back down.

Non-functional requirements

NF1	Buttons are pressed from many threads at once. A press of a hall button must be assigned to exactly one lift, however many people press it at the same moment. (The RUSH command tests this.)
NF2	A new dispatch strategy (for example zoning, where L1 serves floors 0–4) must not change the lift's code.
NF3	A new lift mode (for example FIRE: go to the ground floor and stay open) should be one new class, without rewriting the others.
NF4	A lift must not know who listens to its stops: the hall buttons, a display, an analytics feed later.

Out of scope: capacity and weight, door sensors, acceleration, and using the call's direction to decide where a lift stops (R5 ignores it; Part C 2 asks about it).

Commands and exact output

Command	Output
`BUILDING floors lifts`	`Building ready: floors 0-9, 3 lifts` (`1 lift` for one). Replaces any earlier building; the time starts again at t=0.
`CALL floor UP|DOWN`	`Call 3 DOWN: assigned to L1` · `Call 7 DOWN: already assigned to L2` · `Call 6 DOWN: L1 is already here` · `Call 2 UP: no lift in service, waiting` · `Call 2 UP: already waiting`
`GO lift floor`	`L2 will stop at 8` · `L2 is already at 2` (its doors are open there)
`STEP n`	Runs n ticks. Prints only the announcements, like `t=7: L1 stops at 6`.
`STATUS`	`t=10`, then one line per lift: `L1 at 4, MOVING DOWN, stops [3]` · `L3 at 0, IDLE` · `L2 at 2, DOORS OPEN` · `L1 at 0, MAINTENANCE` (stops only when it has some), then `Waiting: [2 UP, 4 DOWN]` if any are waiting
`MAINT lift ON|OFF`	`L3 is out of service for maintenance` · `L3 is back in service`, then one line per waiting call it dispatched, in the CALL format
`STRATEGY name`	`Strategy: LEAST_TRAVEL`
`RUSH n floor UP|DOWN`	n threads press the same hall button at the same moment. Print only `Rush: 1 assigned, 49 already assigned`.

Error	Output
any command before BUILDING	`No building set up`
a building that's too small	`A building needs at least 2 floors and 1 lift`
a floor that doesn't exist	`Floor 12 does not exist`
UP at the top floor / DOWN at the ground floor	`No UP button at floor 9` / `No DOWN button at floor 0`
an unknown direction	`Unknown direction LEFT`
an unknown lift	`Unknown lift L4`
GO to a lift in maintenance	`L3 is under maintenance`
MAINT ON when not idle, or with stops	`L2 is busy`
MAINT ON twice / MAINT OFF when not in maintenance	`L3 is already under maintenance` / `L1 is not under maintenance`
an unknown strategy	`Unknown strategy FASTEST`

Which error or answer wins? CALL: the direction, then the floor, then the button exists; then lit → waiting → already here → assign. GO: the lift, then the floor, then "already at", then maintenance.

Part A — design

1	The lift's states × its operations (tick, add a stop, maintenance on, maintenance off). Fill the table.
2	There are two separate decisions: **which lift** answers a hall call, and **in what order** a lift visits its stops. Where does each one live? Which one is a Strategy, and why keep them apart?
3	What does a lift announce, and who listens? Why doesn't the lift call them directly?
4	NF1: which steps of a hall call must be one indivisible step? What do you lock? Why is one lock for the whole controller acceptable here?

Part B — code

input.txt

```
CALL 3 UP
BUILDING 10 3
CALL 9 UP
CALL 0 DOWN
CALL 12 DOWN
GO L4 3
GO L1 6
STEP 2
CALL 3 DOWN
STATUS
STEP 3
CALL 3 UP
STRATEGY LEAST_TRAVEL
CALL 2 UP
STRATEGY FASTEST
STEP 2
CALL 6 DOWN
GO L2 2
GO L2 8
STEP 3
STATUS
RUSH 50 7 DOWN
STEP 4
CALL 7 DOWN
MAINT L3 ON
MAINT L3 ON
GO L3 5
MAINT L2 ON
MAINT L1 OFF
MAINT L3 OFF
STATUS
BUILDING 5 1
MAINT L1 ON
CALL 2 UP
CALL 4 DOWN
CALL 2 UP
STATUS
MAINT L1 OFF
STEP 6
STATUS
CALL 4 DOWN
STEP 1
```

expected output

```
No building set up
Building ready: floors 0-9, 3 lifts
No UP button at floor 9
No DOWN button at floor 0
Floor 12 does not exist
Unknown lift L4
L1 will stop at 6
Call 3 DOWN: assigned to L1
t=2
L1 at 2, MOVING UP, stops [3, 6]
L2 at 0, IDLE
L3 at 0, IDLE
t=3: L1 stops at 3
Call 3 UP: assigned to L1
Strategy: LEAST_TRAVEL
Call 2 UP: assigned to L2
Unknown strategy FASTEST
t=7: L1 stops at 6
t=7: L2 stops at 2
Call 6 DOWN: L1 is already here
L2 is already at 2
L2 will stop at 8
t=10
L1 at 4, MOVING DOWN, stops [3]
L2 at 4, MOVING UP, stops [8]
L3 at 0, IDLE
Rush: 1 assigned, 49 already assigned
t=11: L1 stops at 3
t=13: L2 stops at 7
Call 7 DOWN: assigned to L2
L3 is out of service for maintenance
L3 is already under maintenance
L3 is under maintenance
L2 is busy
L1 is not under maintenance
L3 is back in service
t=14
L1 at 3, IDLE
L2 at 7, MOVING UP, stops [7, 8]
L3 at 0, IDLE
Building ready: floors 0-4, 1 lift
L1 is out of service for maintenance
Call 2 UP: no lift in service, waiting
Call 4 DOWN: no lift in service, waiting
Call 2 UP: already waiting
t=0
L1 at 0, MAINTENANCE
Waiting: [2 UP, 4 DOWN]
L1 is back in service
Call 2 UP: assigned to L1
Call 4 DOWN: assigned to L1
t=2: L1 stops at 2
t=5: L1 stops at 4
t=6
L1 at 4, IDLE
Call 4 DOWN: assigned to L1
t=7: L1 stops at 4
```

Reading the output. NEAREST gives `CALL 3 UP` to L1 even though L1 is on its way up to 6; LEAST_TRAVEL then gives `CALL 2 UP` to the idle L2 instead. In the last status of the first building, L2 is at 7 with stops [7, 8]: its doors closed at t=14 and it's moving up, so the new call at 7 is behind it. It will serve 7 on the way back down. RUSH is explained in the solution; if your locking is wrong, it will show 2 assigned only *sometimes*.

Part C — pressure questions

1	**Change.** "Add FIRE mode: when the alarm rings, every lift drops its stops, goes straight to the ground floor, opens its doors and stays there until it's reset."
2	**Change.** "The person at floor 3 pressed DOWN, but L1 picked them up on its way UP. Real lifts don't do that." How would you fix it, and what in your design changes?
3	**Failure.** "L2 breaks down between floors with two hall calls assigned to it."
4	**Concurrency.** "In the real tower, each lift has its own controller thread, and buttons are pressed on other threads. Someone suggests a lock per lift instead of one big lock." What can go wrong? What would you do?
5	**Defend it.** "A State pattern for a lift? An enum and a `switch` in `tick()` would do."

Hints

Read one level at a time, and only after 15 minutes of being stuck. Each level gives away more than the last.

Problem 1 — Deck of Cards

Hint 1 (a nudge). Most of this problem is data that never changes. For each noun, ask: is it a fixed set (enum), a few fields that never change (record), or something whose contents change (class)?

Hint 2 (the shape). `Suit` and `Rank` are enums with fields (a letter; a value and a label). `Card` is a record. `Deck` owns a list of cards. `Hand` works out its type once, in its constructor. The ranking is a separate `Comparator<Hand>`, and MUFLIS is simply that comparator's `.reversed()`.

Hint 3 (the key decision). Sort the three rank values into low, mid, high. Then, in this order: low == high is a trail; three in a row (or exactly A, 2, 3) is a sequence, pure if one suit; one suit is a color; mid equal to a neighbour is a pair (the middle card is *always* in the pair); anything else is a high card. Store a tie-break list, such as [pair rank, other card], and compare it left to right.

Problem 2 — ATM

Hint 1. There are two separate jobs: the session (what you may do depends on how far you've got) and the cash (which notes make up an amount). And a third party, the bank, which the ATM only talks to.

Hint 2. State pattern: IDLE, CARD_INSERTED and AUTHENTICATED, holding no data; the `Atm` holds the card and the cassettes and passes every operation to its state. Chain of Responsibility: each `Cassette` has a note, a count and a link to the next cassette. The wrong-PIN count is a field of `Card`, on the bank's side.

Hint 3. A withdrawal is three steps. (1) Plan on paper: walk the chain and write down how many notes each cassette would give, changing nothing; if anything is left over, refuse. (2) Debit the account: the bank checks and subtracts in one `synchronized` step, or throws. (3) Only then take the notes out of the cassettes.

Problem 3 — Splitwise

Hint 1. What varies is how an amount turns into shares. Everything after that, updating the balances, is the same for every split type.

Hint 2. `SplitStrategy.split(total, entries)` returns a list of shares; one class per type, chosen from a `Map<String, Supplier<SplitStrategy>>`. Money is a `long` of paise, with a parse helper (use `BigDecimal` to read the text) and a format helper. A `Ledger` keeps one net balance per person.

Hint 3. EQUAL and PERCENT are the same calculation with different weights (1 each, or the percentages): share = total × weight / sum of weights, rounded down; then hand out the leftover paise in order. For SIMPLIFY, copy the balances and loop: the biggest debtor pays the biggest creditor the smaller of the two amounts, until nobody is left.

Problem 4 — Elevator System

Hint 1. Keep three things apart: the **lift** (its own movement, tick by tick), the **dispatcher** (which lift gets a hall call), and the **bookkeeping** (lit buttons and the waiting queue). Each one changes for a different reason.

Hint 2. `Lift` with a `LiftState` (IDLE, MOVING, DOORS_OPEN, MAINTENANCE as shared constants) and the data: floor, direction, a `TreeSet` of stops. `DispatchStrategy` with `cost(lift, call)` and a shared `choose()`. Two listeners: `HallButtons` (turns lights off) and `Display` (collects the announcements). `ElevatorSystem` as the front door, with every public method `synchronized`.

Hint 3. The `TreeSet` does the "keep going" work: `higher(floor)` and `lower(floor)` tell you whether there's a stop ahead; `ceiling` and `floor` find the nearest stop either way. In `call()`: check the lit button, choose a lift, add the stop and light the button, all inside the one lock. Give the clock to the display, not to the lift.

Solution 1 — Deck of Cards

Questions a strong candidate asks first

Question	Answer assumed here
Which game's rules? Which hand types, in what order?	Teen Patti: trail, pure sequence, sequence, color, pair, high card
Is the ace high or low?	High, except in A-2-3, the lowest sequence
Do suits ever break a tie?	No; equal hands tie
How do I make a shuffle repeatable for tests?	A seed: `Collections.shuffle(cards, new Random(seed))`
Will other variants be needed?	Yes: MUFLIS now, others later

Part A — model answer

Entity	Kind	Why
`Suit`	enum with a letter	a fixed set of four; the letter is just data
`Rank`	enum with a value and a label	a fixed set of 13: the value is for comparing, the label for printing ("10")
`Card`	record (rank, suit)	two fields that never change. `equals` and `hashCode` come free, and the duplicate check and `deck.take()` rely on them
`Deck`	class	its contents change: shuffled, dealt, taken from
`Hand`	class	a player, three cards, and their evaluation, worked out once
`HandType`	enum, weakest first	a fixed set of six; the declaration order *is* the strength, so `compareTo` compares strength
`TeenPattiRanking`	`Comparator<Hand>`	the rule for "better", kept outside the hand so a variant can swap it
`Table`	class	the round: the deck, the hands at the table, the current variant

Question	Owner	Why
Is this card in the deck?	`Deck`	it holds the cards
What type of hand is this?	`Hand`	it holds the cards. Worked out once, in the constructor, because a hand never changes
Which of two hands is better?	the comparator	a rule about *two* hands, and it varies (R8)

```
Table 1 ◆——— 1 Deck 1 ◆——— 0..52 Card ———> Rank, Suit
Table 1 ◆——— * Hand ———> 3 Card,   Hand ———> HandType
Table ———> «interface» Comparator<Hand>  ◁- - -  TeenPattiRanking   (MUFLIS = the same one, reversed)
```

3. Comparator, not Comparable. `Comparable` means "this class has one natural order". A hand doesn't: under STANDARD a trail is best, under MUFLIS it's worst. Keeping the rule in a `Comparator` means a variant is just another comparator over the same hands (NF1), and MUFLIS costs one line: `new TeenPattiRanking().reversed()`. A Comparator *is* a Strategy: a swappable rule behind a small interface. Java just gave it a name first.

4. Testable shuffling: pass the randomness in. The deck never creates its own `Random`; it receives one (`deck.shuffle(random)`). Tests pass `new Random(42)` and get the same order every time; the real app passes an unseeded one. With no seed in the input, you'd still keep that seam, so the code under test never decides its own randomness.

R4: check everything, then change. The same rule as the vending machine's change: `give()` parses and checks all three cards first, and only then takes them from the deck. A failed HAND leaves the deck exactly as it was.

Patterns used: Comparator as a Strategy; enums and records doing real work. **Deliberately not used:** a class per hand type (the types differ only in data: a strength and a tie-break), State (nothing has modes), a Factory class (a small map of variants is enough), Singleton.

Part B — the code

```
import java.util.*;
import java.util.stream.*;

class CardException extends RuntimeException {
    CardException(String message) { super(message); }
}

enum Suit {
    SPADES('S'), HEARTS('H'), DIAMONDS('D'), CLUBS('C');

    final char letter;
    Suit(char letter) { this.letter = letter; }

    static Optional<Suit> of(char letter) {
        return Arrays.stream(values()).filter(s -> s.letter == letter).findFirst();
    }
}

enum Rank {
    TWO(2, "2"), THREE(3, "3"), FOUR(4, "4"), FIVE(5, "5"), SIX(6, "6"), SEVEN(7, "7"),
    EIGHT(8, "8"), NINE(9, "9"), TEN(10, "10"), JACK(11, "J"), QUEEN(12, "Q"), KING(13, "K"),
    ACE(14, "A");

    final int value;
```

```
    final String label;
    Rank(int value, String label) { this.value = value; this.label = label; }

    static Optional<Rank> of(String label) {
        return Arrays.stream(values()).filter(r -> r.label.equals(label)).findFirst();
    }
}

record Card(Rank rank, Suit suit) {
    static Card parse(String text) {
        if (text.length() >= 2) {
            Optional<Rank> rank = Rank.of(text.substring(0, text.length() - 1));
            Optional<Suit> suit = Suit.of(text.charAt(text.length() - 1));
            if (rank.isPresent() && suit.isPresent()) return new Card(rank.get(), suit.get());
        }
        throw new CardException("Invalid card " + text);
    }

    @Override public String toString() { return rank.label + suit.letter; }
}

// Declared weakest first, so a later constant beats an earlier one.
enum HandType {
    HIGH_CARD("high card"), PAIR("pair"), COLOR("color"),
    SEQUENCE("sequence"), PURE_SEQUENCE("pure sequence"), TRAIL("trail");

    final String label;
    HandType(String label) { this.label = label; }
}

class Hand {
    private final String player;
    private final List<Card> cards;                // in the order received
    private final HandType type;
    private final List<Integer> tieBreak;          // compared left to right; higher wins

    Hand(String player, List<Card> cards) {
        this.player = player;
        this.cards = List.copyOf(cards);
        int[] v = cards.stream().mapToInt(c -> c.rank().value).sorted().toArray();
        int low = v[0], mid = v[1], high = v[2];
        boolean sameSuit = cards.stream().map(Card::suit).distinct().count() == 1;
        boolean aceLow = high == 14 && mid == 3 && low == 2;   // A-2-3
        boolean run = aceLow || (high == mid + 1 && mid == low + 1);

        if (high == low) {
            type = HandType.TRAIL;            tieBreak = List.of(high);
        } else if (run) {
            type = sameSuit ? HandType.PURE_SEQUENCE : HandType.SEQUENCE;
            tieBreak = List.of(aceLow ? 3 : high);             // A-2-3 is the lowest run
        } else if (sameSuit) {
            type = HandType.COLOR;            tieBreak = List.of(high, mid, low);
        } else if (high == mid || mid == low) {
            type = HandType.PAIR;                              // the middle card is in the pair
            tieBreak = List.of(mid, high == mid ? low : high);
        } else {
            type = HandType.HIGH_CARD;        tieBreak = List.of(high, mid, low);
        }
    }

    String player()         { return player; }
    HandType type()         { return type; }
    List<Integer> tieBreak() { return tieBreak; }

    @Override public String toString() {
        String shown = cards.stream().map(Card::toString).collect(Collectors.joining(" "));
        return player + ": " + shown + " (" + type.label + ")";
    }
}

class TeenPattiRanking implements Comparator<Hand> {          // puts the stronger hand first
    public int compare(Hand a, Hand b) {
        int byType = b.type().compareTo(a.type());
        if (byType != 0) return byType;
        for (int i = 0; i < a.tieBreak().size(); i++) {
            int c = Integer.compare(b.tieBreak().get(i), a.tieBreak().get(i));
            if (c != 0) return c;
        }
        return 0;                                                // suits never break a tie
    }
}

class Deck {
    private final List<Card> cards = new ArrayList<>();       // index 0 is the top

    Deck() {
        for (Suit s : Suit.values())
            for (Rank r : Rank.values()) cards.add(new Card(r, s));
    }

    void shuffle(Random random)  { Collections.shuffle(cards, random); }
    int size()                   { return cards.size(); }
    Card dealTop()               { return cards.remove(0); }
    boolean contains(Card card)  { return cards.contains(card); }
    void take(Card card)         { cards.remove(card); }
}

record Showdown(List<Hand> ranked, List<Hand> winners) {}

class Table {
    private static final Map<String, Comparator<Hand>> VARIANTS = Map.of(
```

```
            "STANDARD", new TeenPattiRanking(),
            "MUFLIS", new TeenPattiRanking().reversed());       // lowest hand wins

    private Deck deck = new Deck();
    private final Map<String, Hand> hands = new LinkedHashMap<>();   // in the order players joined
    private Comparator<Hand> ranking = VARIANTS.get("STANDARD");

    void newDeck()                { deck = new Deck(); hands.clear(); }
    void shuffle(long seed)       { deck.shuffle(new Random(seed)); }
    int cardsLeft()               { return deck.size(); }

    void setVariant(String name) {
        Comparator<Hand> chosen = VARIANTS.get(name);
        if (chosen == null) throw new CardException("Unknown variant " + name);
        ranking = chosen;
    }

    List<Hand> deal(List<String> players) {
        Set<String> seen = new HashSet<>();
        for (String p : players) {                              // check everything first...
            if (hands.containsKey(p) || !seen.add(p))
                throw new CardException(p + " already has a hand");
        }
        int need = 3 * players.size();
        if (need > deck.size())
            throw new CardException("Not enough cards: need " + need + ", have " + deck.size());

        Map<String, List<Card>> dealt = new LinkedHashMap<>();   // ...then change
        for (String p : players) dealt.put(p, new ArrayList<>());
        for (int round = 0; round < 3; round++)
            for (String p : players) dealt.get(p).add(deck.dealTop());
        List<Hand> result = new ArrayList<>();
        dealt.forEach((p, cards) -> result.add(seat(new Hand(p, cards))));
        return result;
    }

    Hand give(String player, List<String> cardTexts) {
        if (hands.containsKey(player)) throw new CardException(player + " already has a hand");
        if (cardTexts.size() != 3) throw new CardException("A hand needs exactly 3 cards");
        List<Card> cards = cardTexts.stream().map(Card::parse).toList();
        Set<Card> seen = new HashSet<>();
        for (Card c : cards)
            if (!seen.add(c)) throw new CardException("Duplicate card " + c);
        for (Card c : cards)
            if (!deck.contains(c)) throw new CardException(c + " is not in the deck");
        cards.forEach(deck::take);                              // all checks passed: now change
        return seat(new Hand(player, cards));
    }

    Showdown show() {
        if (hands.size() < 2) throw new CardException("Need at least 2 hands to show");
        List<Hand> ranked = new ArrayList<>(hands.values());
        ranked.sort(ranking);                                   // stable: ties keep the joining order
        List<Hand> winners = ranked.stream()
                .filter(h -> ranking.compare(h, ranked.get(0)) == 0).toList();
        hands.clear();                                          // the round is over
        return new Showdown(ranked, winners);
    }

    private Hand seat(Hand hand) { hands.put(hand.player(), hand); return hand; }
}

public class Main {
    public static void main(String[] args) {
        Table table = new Table();
        Scanner in = new Scanner(System.in);
        while (in.hasNextLine()) {
            String line = in.nextLine().trim();
            if (line.isEmpty()) continue;
            String[] p = line.split("\\s+");
            List<String> rest = Arrays.asList(p).subList(1, p.length);
            try {
                switch (p[0]) {
                    case "NEW_DECK" -> {
                        table.newDeck();
                        System.out.println("New deck: " + table.cardsLeft() + " cards");
                    }
                    case "SHUFFLE" -> {
                        table.shuffle(Long.parseLong(p[1]));
                        System.out.println("Shuffled with seed " + p[1]);
                    }
                    case "DEAL" -> {
                        table.deal(rest).forEach(System.out::println);
                        System.out.println("Cards left: " + table.cardsLeft());
                    }
                    case "HAND" -> System.out.println(table.give(p[1], rest.subList(1, rest.size())));
                    case "SHOW" -> {
                        Showdown s = table.show();
                        s.ranked().forEach(System.out::println);
                        List<String> names = s.winners().stream().map(Hand::player).toList();
                        System.out.println(names.size() == 1 ? "Winner: " + names.get(0)
                                                             : "Tie: " + String.join(", ", names));
                    }
                    case "VARIANT" -> {
                        table.setVariant(p[1]);
                        System.out.println("Variant: " + p[1]);
                    }
                    default -> System.out.println("Unknown command: " + p[0]);
                }
            } catch (CardException e) {
                System.out.println(e.getMessage());
```

```
            }
        }
    }
}
```

How to read it: `Hand`'s constructor checks the types in the order of R5, best first, so the first match wins. The comparator compares the type, then the tie-break list, left to right. `List.sort` is guaranteed stable, which is why tied hands keep the order they joined in. The winners are every hand that compares equal to the first one.

Part C — model answers

1. A-2-3 as the second-best sequence. One line changes: the tie-break of a sequence. Use two numbers: the top card, then a "proper top" flag. Q-K-A gets [14, 1], A-2-3 gets [14, 0], K-Q-J gets [13, 1]: compared left to right, that's Q-K-A, then A-2-3, then K-Q-J. Card, Deck, Table and the comparator don't change. But notice where that line lives: in `Hand`. If variants begin to differ in *how hands are evaluated*, not just in their order, move the evaluation into the variant: a `HandRules` interface with `type(cards)` and `tieBreak(cards)`, and a variant becomes rules plus a comparator. Today it's one line; with a second rule set, extract the interface.

2. Two decks. Several assumptions break. (a) Two AS cards can now legally sit in one hand, so "Duplicate card" is wrong. (b) A pair can now be all one suit (AS AS KS): is it a pair or a color? R5 needs a new rule, because the order of your checks would silently decide. (c) A trail can be three identical cards. (d) `deck.take(card)` still works: `List.remove` removes one copy. The deeper point: a record's `equals` means "same face", not "same physical card". If you ever need to tell physical cards apart, add a deck number to `Card`. What equality means is a design decision.

3. The cousin's shuffle is biased. Swapping each position with *any* position makes 52^{52} equally likely sequences of choices. That number can't be divided evenly among the 52! possible orders of the deck, so some orders come up more often than others. Fisher–Yates is fair: go from the last position down to the second, and swap position i with a random position from 0 to i. That's exactly what `Collections.shuffle` does. Don't write your own.

4. Testing SHOW. Take the randomness out of the test. Build hands from fixed cards, as HAND does, and check the order: trail beats pure sequence, A-2-3 loses to 2-3-4, 9-9-K beats 9-9-5, equal hands tie. Test the comparator on pairs of hands directly. For the deck itself, check facts that hold for any seed: a new deck has 52 different cards, dealing 9 leaves 43, and the same seed gives the same order.

5. "A card is just an int 0–51." "You'd be right if we stored millions of hands or needed raw speed: an int or a byte per card is compact, and I'd convert to one at the edges, for storage or the network. Inside the program, enums make wrong values impossible: there's no rank 15 and no suit 'X' (NF2). The code also reads like the rules, `rank == ACE` instead of `n % 13 == 12`, and the compiler checks every `switch` over a suit. For this app, clarity wins."

Common mistakes

- Building the deck with ranks in the outer loop: the order differs, so the seeded shuffle deals different cards.
- Putting the ranking in `compareTo` inside `Hand`; MUFLIS then needs a flag inside every hand.
- Counting A-2-3 as ace-high (it would beat K-Q-J), or treating K-A-2 as a sequence.
- Taking cards out of the deck while still checking, so a failed HAND loses cards.
- Comparing `Integer` objects with `==`. It works for small numbers by luck (Java caches them), then breaks. Compare `int`s, or use `equals`.
- Re-working the hand type on every comparison instead of once in the constructor.
- Exact text: lower-case type names, and `Tie: Asha, Ravi` with a comma and a space.

Score yourself (1 point each)

☐	Suit, Rank and HandType are enums; Card is a record
☐	The deck is built in the stated order and shuffled with the given seed
☐	A hand works out its type once, in its constructor
☐	A-2-3 is the lowest sequence, Q-K-A the highest, and K-A-2 isn't one
☐	The ranking is a Comparator outside the hand, and MUFLIS is `.reversed()`
☐	HAND checks everything before taking any card
☐	Ties keep the joining order, and a shared top prints `Tie:`
☐	Your output matches exactly
☐	Part C 2: you saw that, with two decks, a pair can be all one suit
☐	Part C 3: you named Fisher–Yates, and why the cousin's shuffle is biased

Solution 2 — ATM

Questions a strong candidate asks first

Question	Answer assumed here
Which notes, and how are they chosen?	Rs 500, 200, 100; largest first
Can one session do several things?	Yes, until EJECT
What happens after wrong PINs? Is the count per session?	Three in a row block the card; the count lives on the card
What are the limits?	Multiples of Rs 100, at most Rs 10,000 each time; no daily limit
Can one account have several cards?	Yes: joint accounts
What if the notes can't make the amount, or the bank says no?	Refuse, and change nothing

Part A — model answer

1. The state table. 18 cells, 18 rules. Each state class is one row.

	insert	PIN	balance	withdraw	eject	load
IDLE	check the card → CARD_INSERTED	"Insert a card first"	"Insert…"	"Insert…"	"Insert…"	allowed
CARD_INSERTED	"already inserted"	right → AUTHENTICATED; wrong → count; third → block, end session	"Enter your PIN first"	"Enter…"	→ IDLE	"during a session"
AUTHENTICATED	"already inserted"	"already accepted"	show it	plan, debit, take	→ IDLE	"during a session"

2. The wrong-PIN count lives on the card. In the ATM or in the session, ejecting and inserting again would reset it, so a thief could try three PINs per insert, forever. On the card (in real life, in the bank's database) it survives sessions and even other ATMs. The ATM also never compares the PIN itself: it asks the card, and the card answers with a `PinCheck`. Who owns the data owns the behaviour, and here that's also the secure choice.

```
Atm ──> Cassette(500) ──next──> Cassette(200) ──next──> Cassette(100) ──next──> null

plan(1700):  500 gives 3 (1500), passes on 200  →  200 gives 1, passes on 0  →  done: {500=3, 200=1}
plan(600), with 500x7 200x3 100x0:  500 gives 1, passes on 100  →  200 gives 0  →  100 gives 0  →  100 left: refuse
```

3. One link's job. Take what I can, `min(count, amount / note)`, write it in the plan, pass the rest to the next link. The last link returns what's left; 0 means the plan works. Planning only writes in a map, so a refused plan changes nothing. Loading is a chain too: each link takes its own note or passes it on, and the last one says "not supported". NF1 holds: a machine with other cassettes is built with a different chain in `main`, and no dispensing code changes.

#	Withdrawal step	If it fails
1	Check the amount and the limit	throw; nothing has changed
2	Plan the notes on paper	throw; nothing has changed
3	Debit the account: check and subtract in one `synchronized` step	throw; nothing has changed. **This is the commit point.**
4	Take the notes from the cassettes, then dispense	past the commit point: a failure here needs compensation (Part C 3)

Entity	Kind	Job
`Atm`	class (the context)	holds the card and the chain; passes every operation to its state
`AtmState` + 3 states	interface + classes	the rules, one row of the table each; no data of their own
`Cassette`	class (a chain link)	a note, a count, the next link
`Bank`	class	accounts and cards; the ATM's only way to reach them
`Account`	class	the balance; `debit` checks and subtracts under one lock (NF3)
`Card`	class	number, account, PIN, the wrong-PIN count, blocked or not
`PinCheck`, `Withdrawal`	records	results handed back

```
Atm ———> «interface» AtmState  <|- - -  IdleState, CardInsertedState, AuthenticatedState
Atm ———> Bank 1 ◆——— * Account          Bank 1 ◆——— * Card ———> 1 Account
Atm ———> Cassette ──next──> Cassette ──next──> Cassette
```

Deliberately not used: a Strategy for choosing notes (there's one rule; Part C 2 might create a second), Observer (nobody reacts to withdrawals yet; an SMS alert would be the first listener), a Singleton class (one ATM, created once in `main`).

Part B — the code

```
import java.util.*;

class AtmException extends RuntimeException {
    AtmException(String message) { super(message); }
}

// ---------------- the bank side: in real life, the ATM reaches it over the network ----------------
class Account {
    final String id;
```

```
    private int balance;

    Account(String id, int balance) { this.id = id; this.balance = balance; }

    synchronized int balance() { return balance; }

    synchronized void debit(int amount) {             // check and subtract as one step
        if (amount > balance) throw new AtmException("Insufficient balance");
        balance -= amount;
    }
}

record PinCheck(boolean ok, int attemptsLeft) {}

class Card {
    private static final int MAX_WRONG = 3;
    final String number;
    final Account account;
    private final String pin;
    private int wrongInARow;                          // lives on the card, so ejecting doesn't reset it
    private boolean blocked;

    Card(String number, Account account, String pin) {
        this.number = number; this.account = account; this.pin = pin;
    }

    boolean isBlocked() { return blocked; }

    PinCheck verify(String attempt) {
        if (pin.equals(attempt)) {
            wrongInARow = 0;
            return new PinCheck(true, MAX_WRONG);
        }
        wrongInARow++;
        if (wrongInARow == MAX_WRONG) blocked = true;
        return new PinCheck(false, MAX_WRONG - wrongInARow);
    }
}

class Bank {
    private final Map<String, Account> accounts = new HashMap<>();
    private final Map<String, Card> cards = new HashMap<>();

    void openAccount(String id, int balance) {
        if (accounts.containsKey(id)) throw new AtmException("Account " + id + " already exists");
        accounts.put(id, new Account(id, balance));
    }

    void issueCard(String number, String accountId, String pin) {
        Account account = accounts.get(accountId);
        if (account == null) throw new AtmException("Unknown account " + accountId);
        if (cards.containsKey(number)) throw new AtmException("Card " + number + " already exists");
        cards.put(number, new Card(number, account, pin));
    }

    Card card(String number) {
        Card card = cards.get(number);
        if (card == null) throw new AtmException("Unknown card " + number);
        return card;
    }
}

// ---------------- the cash: a chain of cassettes, largest note first ----------------
class Cassette {
    final int note;
    private int count;
    private final Cassette next;                      // null for the last link

    Cassette(int note, Cassette next) { this.note = note; this.next = next; }

    void load(int n, int howMany) {                   // my note? keep them; otherwise pass it on
        if (n == note) count += howMany;
        else if (next != null) next.load(n, howMany);
        else throw new AtmException("Rs " + n + " notes are not supported");
    }

    // On paper only: take what I can, write it in the plan, pass the rest on.
    // Returns the part that nobody could pay (0 means the plan works).
    int plan(int amount, Map<Integer, Integer> plan) {
        int use = Math.min(count, amount / note);
        if (use > 0) plan.put(note, use);
        int rest = amount - use * note;
        return (rest == 0 || next == null) ? rest : next.plan(rest, plan);
    }

    void take(Map<Integer, Integer> plan) {
        count -= plan.getOrDefault(note, 0);
        if (next != null) next.take(plan);
    }

    void describe(StringJoiner out) {
        out.add(note + "x" + count);
        if (next != null) next.describe(out);
    }

    int total() { return note * count + (next == null ? 0 : next.total()); }
}

record Withdrawal(int amount, Map<Integer, Integer> notes, int balance) {}

// ---------------- the states ----------------
```

```
interface AtmState {
    void insertCard(Atm atm, String number);
    void enterPin(Atm atm, String pin);
    int balance(Atm atm);
    Withdrawal withdraw(Atm atm, int amount);
    Card eject(Atm atm);
    void checkCanLoad();

    AtmState IDLE = new IdleState();
    AtmState CARD_INSERTED = new CardInsertedState();
    AtmState AUTHENTICATED = new AuthenticatedState();

    static AtmException cardInside() { return new AtmException("A card is already inserted"); }
    static AtmException inSession()  { return new AtmException("Cannot load cash during a session"); }
}

class IdleState implements AtmState {
    public void insertCard(Atm atm, String number) {
        Card card = atm.bank().card(number);
        if (card.isBlocked()) throw new AtmException("Card " + number + " is blocked");
        atm.startSession(card);
    }
    public void enterPin(Atm atm, String pin)       { throw noCard(); }
    public int balance(Atm atm)                     { throw noCard(); }
    public Withdrawal withdraw(Atm atm, int amount) { throw noCard(); }
    public Card eject(Atm atm)                      { throw noCard(); }
    public void checkCanLoad()                      { }                    // allowed
    private AtmException noCard() { return new AtmException("Insert a card first"); }
}

class CardInsertedState implements AtmState {
    public void insertCard(Atm atm, String number) { throw AtmState.cardInside(); }
    public void enterPin(Atm atm, String pin) {
        Card card = atm.card();
        PinCheck check = card.verify(pin);
        if (check.ok()) { atm.setState(AUTHENTICATED); return; }
        if (check.attemptsLeft() == 0) {
            atm.endSession();                                  // clean up first...
            throw new AtmException("Wrong PIN, card " + card.number + " blocked"); // ...then throw
        }
        int left = check.attemptsLeft();
        throw new AtmException("Wrong PIN, " + left + (left == 1 ? " attempt" : " attempts") + " left");
    }
    public int balance(Atm atm)                     { throw pinFirst(); }
    public Withdrawal withdraw(Atm atm, int amount) { throw pinFirst(); }
    public Card eject(Atm atm)                      { return atm.endSession(); }
    public void checkCanLoad()                      { throw AtmState.inSession(); }
    private AtmException pinFirst() { return new AtmException("Enter your PIN first"); }
}

class AuthenticatedState implements AtmState {
    public void insertCard(Atm atm, String number) { throw AtmState.cardInside(); }
    public void enterPin(Atm atm, String pin)       { throw new AtmException("PIN already accepted"); }
    public int balance(Atm atm)                     { return atm.card().account.balance(); }
    public Withdrawal withdraw(Atm atm, int amount) { return atm.dispense(amount); }
    public Card eject(Atm atm)                      { return atm.endSession(); }
    public void checkCanLoad()                      { throw AtmState.inSession(); }
}

// ---------------- the context: holds the data, delegates the rules ----------------
class Atm {
    static final int MAX_PER_WITHDRAWAL = 10_000;
    private final Bank bank;
    private final Cassette cash;                    // the head of the chain
    private AtmState state = AtmState.IDLE;
    private Card card;                              // the card in the slot, if any

    Atm(Bank bank, Cassette cash) { this.bank = bank; this.cash = cash; }

    // operations: each one asks the current state
    void insertCard(String number) { state.insertCard(this, number); }
    void enterPin(String pin)      { state.enterPin(this, pin); }
    int balance()                  { return state.balance(this); }
    Withdrawal withdraw(int amt)   { return state.withdraw(this, amt); }
    Card eject()                   { return state.eject(this); }

    void load(int note, int count) {
        state.checkCanLoad();
        cash.load(note, count);
    }

    String cashReport() {
        StringJoiner notes = new StringJoiner(" ");
        cash.describe(notes);
        return "Cash: " + notes + ", total Rs " + cash.total();
    }

    // helpers the states use
    Bank bank()               { return bank; }
    Card card()               { return card; }
    void setState(AtmState s) { state = s; }
    void startSession(Card c) { card = c; state = AtmState.CARD_INSERTED; }

    Card endSession() {
        Card ejected = card;
        card = null;
        state = AtmState.IDLE;
        return ejected;
    }

    Withdrawal dispense(int amount) {
```

```
        if (amount <= 0 || amount % 100 != 0)
            throw new AtmException("Amount must be a positive multiple of Rs 100");
        if (amount > MAX_PER_WITHDRAWAL)
            throw new AtmException("Maximum Rs " + MAX_PER_WITHDRAWAL + " per withdrawal");
        Map<Integer, Integer> plan = new LinkedHashMap<>();
        if (cash.plan(amount, plan) != 0)                       // 1. plan on paper
            throw new AtmException("Cannot dispense Rs " + amount + " with the notes available");
        card.account.debit(amount);                             // 2. the bank: yes, or throw
        cash.take(plan);                                        // 3. count the notes out
        return new Withdrawal(amount, plan, card.account.balance());
    }
}

public class Main {
    public static void main(String[] args) {
        Bank bank = new Bank();
        Cassette chain = new Cassette(500, new Cassette(200, new Cassette(100, null)));
        Atm atm = new Atm(bank, chain);
        Scanner in = new Scanner(System.in);
        while (in.hasNextLine()) {
            String line = in.nextLine().trim();
            if (line.isEmpty()) continue;
            String[] p = line.split("\\s+");
            try {
                switch (p[0]) {
                    case "ADD_ACCOUNT" -> {
                        bank.openAccount(p[1], Integer.parseInt(p[2]));
                        System.out.println("Account " + p[1] + " opened with Rs " + p[2]);
                    }
                    case "ADD_CARD" -> {
                        bank.issueCard(p[1], p[2], p[3]);
                        System.out.println("Card " + p[1] + " issued for " + p[2]);
                    }
                    case "LOAD" -> {
                        atm.load(Integer.parseInt(p[1]), Integer.parseInt(p[2]));
                        System.out.println("Loaded Rs " + p[1] + " x " + p[2]);
                    }
                    case "CASH" -> System.out.println(atm.cashReport());
                    case "INSERT" -> {
                        atm.insertCard(p[1]);
                        System.out.println("Card " + p[1] + " inserted, enter PIN");
                    }
                    case "PIN" -> { atm.enterPin(p[1]); System.out.println("PIN accepted"); }
                    case "BALANCE" -> System.out.println("Balance: Rs " + atm.balance());
                    case "WITHDRAW" -> {
                        Withdrawal w = atm.withdraw(Integer.parseInt(p[1]));
                        StringJoiner notes = new StringJoiner(" ");
                        w.notes().forEach((note, n) -> notes.add(note + "x" + n));
                        System.out.println("Dispensed Rs " + w.amount() + " (" + notes + ")"
                                + ", balance Rs " + w.balance());
                    }
                    case "EJECT" -> System.out.println("Card " + atm.eject().number + " ejected");
                    default -> System.out.println("Unknown command: " + p[0]);
                }
            } catch (AtmException e) {
                System.out.println(e.getMessage());
            }
        }
    }
}
```

How to read it: each state class reads as one row of the table. All the money logic sits in `Atm.dispense()`, in the order of the step table. Look at the blocked-card path in `CardInsertedState`: it ends the session first and throws second, the same "clean up, then throw" as the vending machine's refund.

Part C — model answers

1. Deposits. A new operation means a new method in every state: IDLE and CARD_INSERTED refuse, AUTHENTICATED accepts. The compiler lists every state that needs it, so none is forgotten. The cassettes get an `accept(note, count)` that travels down the chain like `load`; a note with no cassette goes back to the customer. Order: count the notes, put them away, then credit the account; if the credit fails, give the notes back. Classes touched: the interface and its three states, `Cassette`, `Account` (a `credit`), the driver. That's the State trade-off from the vending machine: a new **state** is one class, but a new **operation** touches every state.

2. When largest-first fails. Let each link try fewer notes when the rest of the chain can't finish: start with the most it can give, and step down to zero. The chain stays; only the link's rule changes.

```
boolean plan(int amount, Map<Integer, Integer> plan) {
    for (int use = Math.min(count, amount / note); use >= 0; use--) {
        if (use > 0) plan.put(note, use);                // try this many...
        int rest = amount - use * note;
        if (rest == 0 || (next != null && next.plan(rest, plan))) return true;
        plan.remove(note);                               // ...it didn't work: undo, try one fewer
    }
    return false;
}
```

For Rs 600 with one 500 and three 200s: the 500 link tries 1 note, and the rest of the chain can't pay 100, so it undoes that and tries 0; the 200 link then pays 600 with three notes. Run against the test above, `WITHDRAW 600` gives `(200x3)`.
A real bug, caught by running it. The first draft of this snippet wrote each entry *after* the rest of the chain succeeded. The deeper links then wrote first, and the output said `(200x1 500x3)`, the wrong order. Writing first and undoing on failure keeps the largest-first order. Always run your Part C code too.

3. The dispenser jams after the debit. The money has left the account but isn't in the customer's hand. (a) Before dispensing, record the withdrawal as a transaction with a status: PENDING, then DISPENSED or REVERSED. (b) On a jam: reverse the debit (credit the money back: this is **compensation**), mark the transaction REVERSED, take the ATM out of service, alert the bank. (c) If the ATM crashes before it can reverse, the PENDING record is how an end-of-day check, comparing cassette counts with transactions, finds the problem and refunds the customer. Never leave "debited but not dispensed" unrecorded.

4. The joint account at two ATMs. `Account.debit` is `synchronized` and does the check and the subtraction as one step, so one withdrawal gets Rs 4,000 and the other gets "Insufficient balance". Without that, both could check Rs 5,000 ≥ Rs 4,000, both subtract, and the balance would end at −3,000. With three bank servers, `synchronized` protects only one JVM. Move the check into the database: `UPDATE account SET balance = balance - 4000 WHERE id = 'S1' AND balance >= 4000`, and see whether one row changed. Or use a version column (optimistic locking) and retry.

5. "A chain for three notes? A loop is simpler." "You'd be right: with three cassettes that all follow the same rule, a loop over a list is just as clear, and I'd happily start there. The chain earns its place when the links start to differ: a faulty cassette that skips itself, a Rs 2,000 cassette used only for big amounts, or the backtracking rule from the change question living inside each link. The interviewer asked for Chain of Responsibility, so I showed it; I'd pick the loop until a link needs behaviour of its own."

Common mistakes

- Debiting the account before checking the notes: a refused withdrawal leaves the customer short.
- Reducing the cassette counts while planning: a failed plan leaves the counts wrong.
- The wrong-PIN count in the ATM or the session, so ejecting resets it.
- Throwing "card blocked" before ending the session: the next command thinks a card is still inside.
- Checking the balance and debiting in two separate steps: a check-then-act race between ATMs.
- Allowing LOAD during a session.
- Exact text: "1 attempt left", not "1 attempts left".

Score yourself (1 point each)

☐	You drew the 3 × 6 state table before coding
☐	One class per state, holding no data; the ATM holds the card and the cash
☐	The wrong-PIN count lives on the card, and a correct PIN resets it
☐	The blocked path ends the session before throwing
☐	The cassettes form a chain; each link takes what it can and passes the rest on
☐	Planning changes nothing; notes are taken only after the debit succeeds
☐	The debit checks and subtracts in one `synchronized` step
☐	The errors come out in the stated order (amount, limit, notes, balance)
☐	Your output matches exactly
☐	Part C 3: you described compensation and a transaction record

Solution 3 — Splitwise

Questions a strong candidate asks first

Question	Answer assumed here
Which split types?	EQUAL, EXACT, PERCENT; more later
Rupees with paise? How do we round?	Up to 2 decimals; round down, leftover paise to the first listed
Do we need who-owes-whom, or each person's total?	Totals; SIMPLIFY works out the payments
Groups? Other currencies?	Out of scope
Can expenses be edited or deleted?	Not yet (Part C 2)

Part A — model answer

Entity	Kind	Job
`SplitService`	class (front door)	checks an expense completely, then records it
`SplitStrategy` + `EqualSplit`, `ExactSplit`, `PercentSplit`	interface + classes	how an amount turns into shares, and what each type accepts
`Part`	record	one entry as typed: a name, and maybe a value
`Share`	record	a person and their share in paise
`Ledger`	class	one net balance per person; record, settle, simplify
`Expense`, `Transfer`	records	results for the driver to print
`Money`, `Shares`	helpers (static methods)	parse and format paise; the rounding rule of R5

```
SplitService ———> Ledger   (net: Map<String, Long>, in join order)
SplitService ———> «interface» SplitStrategy  ◁- - -  EqualSplit, ExactSplit, PercentSplit
                  (chosen by name from a Map<String, Supplier<SplitStrategy>>)
EqualSplit, PercentSplit ———> Shares.byWeight(...)        all money ———> Money (long paise)
```

1–2. Strategy + Factory. The split types differ in **behaviour** (what entries they accept, what they check, how they compute), not just in numbers, so they're classes. They're chosen by name from a map of suppliers. SHARES is one new class plus one line in the map; no other file changes (NF1). Checks that are the same for every type (the people exist, nobody twice) live once, in the service.

3. One net number per person. **Net balances:** simple, always add up to zero, and exactly what SIMPLIFY needs; but they can't answer "how much do I owe Ravi in particular?". **A pair table** (who owes whom, for every pair): answers that, and matches how people remember things ("I owe Ravi for the pizza"); but both sides of every pair must be kept in step, and it grows with the square of the number of people. Choose by the question you must answer. Here the question is "where does each person stand?", so: net.

4. Rounding without `double`. Rs 100 is 10,000 paise. Divided by 3 that's 3,333 each, with 1 paisa left, which goes to the first person: 3,334, 3,333, 3,333. With `double`, 0.1 + 0.2 is 0.30000000000000004; over hundreds of expenses the errors pile up and balances stop adding to zero. Read text with `BigDecimal`: `new BigDecimal("450.50").movePointRight(2).longValueExact()` gives 45,050, and it throws for `12.345`, which is exactly the check R3 wants.

R6: check everything, then change. The same rule as the vending machine and the ATM. `addExpense` runs every check first, and calls `ledger.record()` last, so a refused expense changes nothing.

Patterns used: Strategy, Factory (a map of suppliers), Facade (`SplitService`). **Deliberately not used:** Template Method (EQUAL and PERCENT share one helper method, which is simpler than a base class), Observer (no notifications yet; "tell Ravi when Asha adds an expense" would be the first listener), Singleton.

Part B — the code

```
import java.math.BigDecimal;
import java.util.*;
import java.util.function.Supplier;

class SplitException extends RuntimeException {
    SplitException(String message) { super(message); }
}

final class Money {                                  // money is paise in a long: never a double
    private Money() {}

    static long parse(String text) {
        try {
            long paise = new BigDecimal(text).movePointRight(2).longValueExact();
            if (paise > 0) return paise;
        } catch (ArithmeticException | NumberFormatException e) {
            // more than 2 decimals, or not a number: fall through
        }
        throw new SplitException("Invalid amount " + text);
    }

    static String format(long paise) { return String.format("%d.%02d", paise / 100, paise % 100); }
}

record Part(String user, String value) {             // "Ravi" -> value null; "Ravi:300" -> value "300"
    static Part parse(String token) {
        int colon = token.indexOf(':');
        return colon < 0 ? new Part(token, null)
```

```
                        : new Part(token.substring(0, colon), token.substring(colon + 1));
    }
    String token() { return value == null ? user : user + ":" + value; }
}

record Share(String user, long paise) {}

// ---------------- the split rules: one class per type ----------------
interface SplitStrategy {
    List<Share> split(long total, List<Part> parts);  // throws if the entries don't fit this type
}

final class Shares {
    private Shares() {}

    // Split in proportion to the weights: round every share down, then hand out
    // the leftover paise one each, in the order listed.
    static List<Share> byWeight(long total, List<Part> parts, long[] weights) {
        long sumOfWeights = Arrays.stream(weights).sum();
        long[] paise = new long[weights.length];
        long given = 0;
        for (int i = 0; i < paise.length; i++) {
            paise[i] = total * weights[i] / sumOfWeights;
            given += paise[i];
        }
        for (int i = 0; given < total; i++, given++) paise[i]++;
        List<Share> shares = new ArrayList<>();
        for (int i = 0; i < paise.length; i++) shares.add(new Share(parts.get(i).user(), paise[i]));
        return shares;
    }

    static SplitException badEntry(Part part, String type) {
        return new SplitException("Bad entry " + part.token() + " for " + type);
    }
}

class EqualSplit implements SplitStrategy {
    public List<Share> split(long total, List<Part> parts) {
        long[] weights = new long[parts.size()];
        for (int i = 0; i < weights.length; i++) {
            if (parts.get(i).value() != null) throw Shares.badEntry(parts.get(i), "EQUAL");
            weights[i] = 1;
        }
        return Shares.byWeight(total, parts, weights);
    }
}

class ExactSplit implements SplitStrategy {
    public List<Share> split(long total, List<Part> parts) {
        List<Share> shares = new ArrayList<>();
        long sum = 0;
        for (Part p : parts) {
            if (p.value() == null) throw Shares.badEntry(p, "EXACT");
            long paise = Money.parse(p.value());
            shares.add(new Share(p.user(), paise));
            sum += paise;
        }
        if (sum != total)
            throw new SplitException("Exact amounts add up to Rs " + Money.format(sum)
                    + ", not Rs " + Money.format(total));
        return shares;
    }
}

class PercentSplit implements SplitStrategy {
    public List<Share> split(long total, List<Part> parts) {
        long[] percents = new long[parts.size()];
        for (int i = 0; i < percents.length; i++) {
            Part p = parts.get(i);
            if (p.value() == null || !p.value().matches("\\d{1,3}"))
                throw Shares.badEntry(p, "PERCENT");
            percents[i] = Long.parseLong(p.value());
        }
        long sum = Arrays.stream(percents).sum();
        if (sum != 100) throw new SplitException("Percentages add up to " + sum + ", not 100");
        return Shares.byWeight(total, parts, percents);
    }
}

// ---------------- balances ----------------
record Transfer(String from, String to, long paise) {}

class Ledger {
    // + means this person is owed money, - means they owe. Kept in the order people joined.
    private final Map<String, Long> net = new LinkedHashMap<>();

    synchronized void addUser(String name) {
        if (net.containsKey(name)) throw new SplitException("User " + name + " already exists");
        net.put(name, 0L);
    }

    synchronized boolean hasUser(String name) { return net.containsKey(name); }

    synchronized void record(String payer, long total, List<Share> shares) {   // one expense: one step
        net.merge(payer, total, Long::sum);
        for (Share s : shares) net.merge(s.user(), -s.paise(), Long::sum);
    }

    synchronized void settle(String from, String to, long paise) {
        net.merge(from, paise, Long::sum);
        net.merge(to, -paise, Long::sum);
```

```
    }

    synchronized Map<String, Long> balances() { return new LinkedHashMap<>(net); }

    // Greedy: the one who owes most pays the one owed most, until everyone is at zero.
    synchronized List<Transfer> simplify() {
        Map<String, Long> left = new LinkedHashMap<>(net);
        List<Transfer> plan = new ArrayList<>();
        while (true) {
            String debtor = extreme(left, -1), creditor = extreme(left, +1);
            if (debtor == null || creditor == null) return plan;
            long paise = Math.min(-left.get(debtor), left.get(creditor));
            plan.add(new Transfer(debtor, creditor, paise));
            left.merge(debtor, paise, Long::sum);
            left.merge(creditor, -paise, Long::sum);
        }
    }

    // sign -1: who owes the most; sign +1: who is owed the most. A tie goes to whoever joined first.
    private static String extreme(Map<String, Long> m, int sign) {
        String best = null;
        for (Map.Entry<String, Long> e : m.entrySet()) {
            long v = e.getValue() * sign;
            if (v > 0 && (best == null || v > m.get(best) * sign)) best = e.getKey();
        }
        return best;
    }
}

record Expense(String payer, long total, String type, List<Share> shares) {}

// ---------------- the front door ----------------
class SplitService {
    private static final Map<String, Supplier<SplitStrategy>> TYPES = Map.of(
            "EQUAL", EqualSplit::new,
            "EXACT", ExactSplit::new,
            "PERCENT", PercentSplit::new);

    private final Ledger ledger = new Ledger();

    void addUser(String name) { ledger.addUser(name); }

    Expense addExpense(String payer, String amount, String type, List<Part> parts) {
        requireUser(payer);
        long total = Money.parse(amount);
        Supplier<SplitStrategy> maker = TYPES.get(type);
        if (maker == null) throw new SplitException("Unknown split type " + type);
        if (parts.isEmpty()) throw new SplitException("Nobody to split with");
        Set<String> seen = new HashSet<>();
        for (Part p : parts) {
            requireUser(p.user());
            if (!seen.add(p.user())) throw new SplitException(p.user() + " is listed twice");
        }
        List<Share> shares = maker.get().split(total, parts);     // may throw: nothing has changed yet
        ledger.record(payer, total, shares);                      // every check passed: now change
        return new Expense(payer, total, type, shares);
    }

    long settle(String from, String to, String amount) {
        requireUser(from);
        requireUser(to);
        if (from.equals(to)) throw new SplitException("Cannot settle with yourself");
        long paise = Money.parse(amount);
        ledger.settle(from, to, paise);
        return paise;
    }

    Map<String, Long> balances() { return ledger.balances(); }
    List<Transfer> simplify()    { return ledger.simplify(); }

    private void requireUser(String name) {
        if (!ledger.hasUser(name)) throw new SplitException("Unknown user " + name);
    }
}

public class Main {
    public static void main(String[] args) {
        SplitService service = new SplitService();
        Scanner in = new Scanner(System.in);
        while (in.hasNextLine()) {
            String line = in.nextLine().trim();
            if (line.isEmpty()) continue;
            String[] p = line.split("\\s+");
            try {
                switch (p[0]) {
                    case "ADD_USER" -> { service.addUser(p[1]); System.out.println("Added " + p[1]); }
                    case "EXPENSE" -> {
                        List<Part> parts = new ArrayList<>();
                        for (int i = 4; i < p.length; i++) parts.add(Part.parse(p[i]));
                        Expense e = service.addExpense(p[1], p[2], p[3], parts);
                        StringJoiner shares = new StringJoiner(", ");
                        e.shares().forEach(s -> shares.add(s.user() + " " + Money.format(s.paise())));
                        System.out.println(e.payer() + " paid Rs " + Money.format(e.total())
                                + ", split " + e.type() + ": " + shares);
                    }
                    case "SETTLE" -> {
                        long paise = service.settle(p[1], p[2], p[3]);
                        System.out.println(p[1] + " paid " + p[2] + " Rs " + Money.format(paise));
                    }
                    case "BALANCES" -> service.balances().forEach((user, v) -> System.out.println(
                            v > 0 ? user + " gets back Rs " + Money.format(v)
```

```
                        : v < 0 ? user + " owes Rs " + Money.format(-v)
                        :         user + " is settled up"));
                    case "SIMPLIFY" -> {
                        List<Transfer> plan = service.simplify();
                        if (plan.isEmpty()) System.out.println("Everyone is settled up");
                        plan.forEach(t -> System.out.println(
                                t.from() + " pays " + t.to() + " Rs " + Money.format(t.paise())));
                    }
                    default -> System.out.println("Unknown command: " + p[0]);
                }
            } catch (SplitException e) {
                System.out.println(e.getMessage());
            }
        }
    }
}
```

How to read it: `addExpense` follows R6 line by line. `Shares.byWeight` is the whole of R5: EQUAL passes weights of 1, PERCENT passes the percentages. `simplify()` works on a copy, so it never touches the real balances. `extreme()` replaces its choice only on a strictly larger amount, which is what sends ties to whoever joined first.

Part C — model answers

1. SHARES. One class, plus `"SHARES", SharesSplit::new` in the map. This was run: `EXPENSE Asha 1000 SHARES Asha:2 Ravi:1 Meera:1` gives 500.00, 250.00, 250.00.

```
class SharesSplit implements SplitStrategy {
    public List<Share> split(long total, List<Part> parts) {
        long[] weights = new long[parts.size()];
        for (int i = 0; i < weights.length; i++) {
            Part p = parts.get(i);
            if (p.value() == null || !p.value().matches("\\d{1,3}") || Long.parseLong(p.value()) == 0)
                throw Shares.badEntry(p, "SHARES");
            weights[i] = Long.parseLong(p.value());
        }
        return Shares.byWeight(total, parts, weights);
    }
}
```

Nothing else changes: not the other strategies, not the ledger, not the driver.

2. Deleting an expense. Today only the totals are kept, so an expense can't be taken back: you must start storing each expense (an ID, the payer, the total, the shares). Then a delete applies it in reverse: the payer goes down by the total, and each person goes up by their share. Or go further: store **only** the expenses and settlements, and work out the balances when asked ("compute, don't store"). That's always right after deletes and edits, and costs one pass over the list per BALANCES, which is nothing for a flat of four. A middle way keeps both, and rebuilds the balances from the list whenever they're in doubt.

3. Two expenses at the same moment. Recording one expense changes several balances. Two threads inside `record()` together can lose an update: both read Ravi's −300, and each writes its own result. A `ConcurrentHashMap` makes each single `merge` safe, but an expense is several merges that belong together; another thread could still see half an expense, and then the balances don't add up to zero. So the whole of `record()` is one `synchronized` step, and so are `balances()` and `simplify()`, so they always see whole expenses. With several servers: a database transaction around saving the expense and updating the balances, or save only the expense and compute balances with a `SUM` query.

4. Not always the fewest payments. For those balances (run while preparing this PDF), the greedy gives four payments: Kiran → Asha 400, Neha → Ravi 300, Kiran → Meera 200, Neha → Meera 100. Three are enough: Neha → Asha 400, Kiran → Ravi 300, Kiran → Meera 300. The true minimum means finding as many groups as possible whose balances add up to zero, and that's NP-hard in general. The greedy promises at most n − 1 payments, which is good enough in practice. For a trip of under about fifteen people, trying every grouping is still feasible if the fewest payments really matter.

5. "A switch is twenty lines." "You'd be right for two types that differ only in a formula. These three differ in what they accept, what they check and how they compute, and a fourth, SHARES, is already wanted. With a switch, each new type edits `addExpense` and its tests, and a type's rules end up spread across the method. With a class per type, each rule set is small, sits in one place and can be tested alone, and adding one touches nothing else."

Common mistakes

- `double` for money. Printing with `%.2f` hides the drift until the balances stop adding up to zero.
- Rounding each share on its own (`Math.round`), so the shares add up to 999.99 or 1000.01.
- Changing balances while still checking: a failed EXACT check leaves the payer credited.
- Forgetting the payer can be one of the people sharing, and gets a share like anyone else.
- Not catching a name listed twice, so that person gets two shares.
- SIMPLIFY changing the real balances instead of a copy.
- Exact text: amounts always have two decimals (`Rs 1200.00`); "is settled up".

Score yourself (1 point each)

☐	One strategy class per split type, chosen from a map by name
☐	Money is a `long` in paise, and parsing rejects more than 2 decimals
☐	EQUAL and PERCENT round down, then hand out the leftover paise in order
☐	Every check happens before the ledger changes
☐	The payer can also be one of the people sharing
☐	Balances are one net number per person, shown in join order
☐	SIMPLIFY works on a copy, and ties go to whoever joined first
☐	Your output matches exactly
☐	Part C 3: you explained why a `ConcurrentHashMap` alone isn't enough
☐	Part C 4: you know the greedy isn't always the minimum, and why

Solution 4 — Elevator System

Questions a strong candidate asks first

Question	Answer assumed here
How many floors and lifts? Does every lift serve every floor?	10 and 3 here; yes (zoning would be a new strategy)
How do we model time?	Ticks: one floor per tick; doors open for one tick
Does a lift stop for a hall call going the other way?	Yes, here: a simplification (Part C 2)
How is a lift chosen? Will that change?	By a strategy the operator can switch
What if every lift is out of service?	Calls wait in a queue
Can many people press buttons at once?	Yes: each hall button is assigned exactly once

Part A — model answer

1. The state table.

	tick	add a stop	maintenance on	maintenance off
IDLE	nothing; or open here; or head for the nearest stop and move	add	allowed with no stops, else "busy"	"not under maintenance"
MOVING	move one floor; open if it's a stop	add	"busy"	"not under…"
DOORS_OPEN	close; then IDLE, or MOVING (keep or turn the direction)	add (its own floor: nothing)	"busy"	"not under…"
MAINTENANCE	nothing	"under maintenance"	"already under…"	→ IDLE

Decision	Lives in	Why there
Which lift answers a hall call?	`DispatchStrategy`, used by the system: **Strategy + Factory**	it varies (NEAREST, LEAST_TRAVEL, zoning later) and is switched by name at run time
In what order does a lift visit its stops?	the lift's own states	it's how a lift moves: keep going while there are stops ahead, then turn around

2. Two decisions, two homes. A new dispatch rule must not touch how a lift moves (NF2), and a better way of moving (Part C 2) must not touch dispatch. If the lift chose which hall calls to accept, both would live in one class and change together.

3. What the lift announces. "I stopped at floor f" (`onStop`). `HallButtons` turns that floor's lit buttons off; `Display` prints the line. The lift knows neither (NF4), so an analytics feed later is one more listener. One honest nuance: turning a button off is **required** bookkeeping, not just an announcement. That's fine here, because it runs inside the same lock and can't fail; a slow listener, like sending to a cloud dashboard, would get a queue instead.

4. NF1: one indivisible step. In a hall call: "is the button lit?" + choose a lift + add the stop + light the button. If these can be separated, two threads both see "not lit" and two lifts come. One lock around the whole controller (every public method `synchronized`) is fine here: a building has a handful of lifts and a few presses a second, and each operation takes microseconds. It's simpler than a lock per lift, and it can't deadlock (Part C 4).

Entity	Kind	Job
`ElevatorSystem`	class (front door)	lifts, lit buttons, waiting queue, strategy; one lock. It also keeps the lifts from talking to each other, which is the Mediator idea
`Lift`	class (the context)	floor, direction, stops (a `TreeSet`), current state
`LiftState` + 4 states	interface + classes	the rules, one row of the table each
`DispatchStrategy` + 2 strategies	interface + classes	a cost for a lift and a call; a shared `choose()`
`HallButtons`, `Display`	listeners	lit buttons; announcements
`Clock`	class	the shared tick count, read by the display
`HallCall`, `CallResult`	records	a button press; the outcome of a call
`Direction`, `CallStatus`	enums	UP, DOWN, NONE; the five outcomes of a call

```
ElevatorSystem 1 ◆——— 1..* Lift ———> «interface» LiftState  ◁- - -  Idle, Moving, DoorsOpen, Maintenance
ElevatorSystem ———> «interface» DispatchStrategy  ◁- - -  NearestLift, LeastTravel
ElevatorSystem ———> HallButtons (lit: Map<HallCall, Lift>)        ElevatorSystem ———> Deque<HallCall> waiting
Lift ———> * «interface» LiftListener  ◁- - -  HallButtons, Display ———> Clock
```

Deliberately not used: Command for requests (a `HallCall` record is enough; Command earns its place with undo or a saved queue), a Singleton class (BUILDING creates one system and the driver holds it).

Why does L2 show stops [7, 8] while it's at floor 7? Its doors closed at t=14 and it's moving up. A moving lift can't stop at the floor it's leaving, so LEAST_TRAVEL treats 7 as behind it (the call isn't *strictly* ahead), and L2 will serve 7 on the way back down. A real lift would probably reopen its doors; this model keeps one tick simple.

Testing a race, honestly. While preparing this PDF, the solution was run with `synchronized` removed from `call()`. The RUSH test showed "2 assigned" in **7 runs out of 40**, and passed in the other 33. As with the parking lot: a race test that passes proves little. The argument in answer 4 is what makes the code correct.

Part B — the code

```
import java.util.*;
import java.util.concurrent.*;
import java.util.function.Supplier;

class LiftException extends RuntimeException {
    LiftException(String message) { super(message); }
}

enum Direction {
    UP(1), DOWN(-1), NONE(0);

    final int step;
    Direction(int step) { this.step = step; }

    Direction opposite() { return this == UP ? DOWN : this == DOWN ? UP : NONE; }

    static Direction parse(String text) {             // a hall button is only ever UP or DOWN
        return switch (text) {
            case "UP" -> UP;
            case "DOWN" -> DOWN;
            default -> throw new LiftException("Unknown direction " + text);
        };
    }
}

record HallCall(int floor, Direction direction) {
    @Override public String toString() { return floor + " " + direction; }
}

interface LiftListener {
    void onStop(Lift lift, int floor);
}

class Clock {
    private int now;
    int now()   { return now; }
    void tick() { now++; }
}

// ---------------- the lift's states ----------------
interface LiftState {
    void tick(Lift lift);
    void addStop(Lift lift, int floor);
    void maintenanceOn(Lift lift);
    void maintenanceOff(Lift lift);
    String label(Lift lift);

    LiftState IDLE = new IdleState();
    LiftState MOVING = new MovingState();
    LiftState DOORS_OPEN = new DoorsOpenState();
    LiftState MAINTENANCE = new MaintenanceState();

    static LiftException busy(Lift lift) { return new LiftException(lift.id + " is busy"); }
    static LiftException notInMaintenance(Lift lift) {
        return new LiftException(lift.id + " is not under maintenance");
    }
}

class IdleState implements LiftState {
    public void tick(Lift lift) {
        if (!lift.hasStops()) return;
        if (lift.removeStopHere()) { lift.openDoors(); return; }   // called where it stands
        lift.setDirection(lift.towardNearestStop());
        lift.setState(MOVING);
        lift.moveOneFloor();
    }
    public void addStop(Lift lift, int floor) { lift.recordStop(floor); }
    public void maintenanceOn(Lift lift) {
        if (lift.hasStops()) throw LiftState.busy(lift);
        lift.setState(MAINTENANCE);
    }
    public void maintenanceOff(Lift lift) { throw LiftState.notInMaintenance(lift); }
    public String label(Lift lift)        { return "IDLE"; }
}

class MovingState implements LiftState {
    public void tick(Lift lift)               { lift.moveOneFloor(); }
    public void addStop(Lift lift, int floor) { lift.recordStop(floor); }
    public void maintenanceOn(Lift lift)      { throw LiftState.busy(lift); }
    public void maintenanceOff(Lift lift)     { throw LiftState.notInMaintenance(lift); }
    public String label(Lift lift)            { return "MOVING " + lift.direction(); }
}

class DoorsOpenState implements LiftState {
    public void tick(Lift lift) {                      // the doors close, then the lift decides
        if (!lift.hasStops()) {
            lift.setDirection(Direction.NONE);
            lift.setState(IDLE);
            return;
        }
        if (lift.direction() == Direction.NONE) lift.setDirection(lift.towardNearestStop());
        else if (!lift.hasStopAhead()) lift.setDirection(lift.direction().opposite());
        lift.setState(MOVING);
    }
    public void addStop(Lift lift, int floor) {
        if (floor != lift.floor()) lift.recordStop(floor);      // its doors are already open here
    }
    public void maintenanceOn(Lift lift)  { throw LiftState.busy(lift); }
    public void maintenanceOff(Lift lift) { throw LiftState.notInMaintenance(lift); }
```

```
    public String label(Lift lift)        { return "DOORS OPEN"; }
}

class MaintenanceState implements LiftState {
    public void tick(Lift lift) { }
    public void addStop(Lift lift, int floor) {
        throw new LiftException(lift.id + " is under maintenance");
    }
    public void maintenanceOn(Lift lift) {
        throw new LiftException(lift.id + " is already under maintenance");
    }
    public void maintenanceOff(Lift lift) { lift.setState(IDLE); }
    public String label(Lift lift)        { return "MAINTENANCE"; }
}

// ---------------- the lift: holds the data, its state holds the rules ----------------
class Lift {
    final String id;
    private int floor = 0;                                  // every lift starts at the ground floor
    private Direction direction = Direction.NONE;
    private final TreeSet<Integer> stops = new TreeSet<>();
    private LiftState state = LiftState.IDLE;
    private final List<LiftListener> listeners;

    Lift(String id, List<LiftListener> listeners) { this.id = id; this.listeners = listeners; }

    // operations: each one asks the current state
    void tick()             { state.tick(this); }
    void addStop(int floor) { state.addStop(this, floor); }
    void maintenanceOn()    { state.maintenanceOn(this); }
    void maintenanceOff()   { state.maintenanceOff(this); }

    // what the dispatcher may read
    int floor()                { return floor; }
    Direction direction()      { return direction; }
    boolean inService()        { return state != LiftState.MAINTENANCE; }
    boolean doorsOpenAt(int f) { return state == LiftState.DOORS_OPEN && floor == f; }

    int farthestStopAhead() {                               // its own floor if there is none
        if (stops.isEmpty()) return floor;
        if (direction == Direction.UP)   return Math.max(floor, stops.last());
        if (direction == Direction.DOWN) return Math.min(floor, stops.first());
        return floor;
    }

    String status() {
        String s = id + " at " + floor + ", " + state.label(this);
        return stops.isEmpty() ? s : s + ", stops " + stops;
    }

    // helpers the states use
    void setState(LiftState s)     { state = s; }
    void setDirection(Direction d) { direction = d; }
    void recordStop(int f)         { stops.add(f); }
    boolean hasStops()             { return !stops.isEmpty(); }
    boolean removeStopHere()       { return stops.remove(floor); }

    boolean hasStopAhead() {
        return direction == Direction.UP ? stops.higher(floor) != null : stops.lower(floor) != null;
    }

    Direction towardNearestStop() {                         // a tie goes up
        Integer up = stops.ceiling(floor), down = stops.floor(floor);
        if (down == null) return Direction.UP;
        if (up == null) return Direction.DOWN;
        return up - floor <= floor - down ? Direction.UP : Direction.DOWN;
    }

    void moveOneFloor() {
        floor += direction.step;
        if (stops.remove(floor)) openDoors();
    }

    void openDoors() {
        state = LiftState.DOORS_OPEN;
        for (LiftListener l : listeners) l.onStop(this, floor);
    }
}

// ---------------- which lift answers a hall call ----------------
interface DispatchStrategy {
    int cost(Lift lift, HallCall call);                     // lower is better

    default Lift choose(HallCall call, List<Lift> available) {
        Lift best = available.get(0);                       // a tie keeps the lower lift number
        for (Lift l : available)
            if (cost(l, call) < cost(best, call)) best = l;
        return best;
    }
}

class NearestLift implements DispatchStrategy {
    public int cost(Lift lift, HallCall call) { return Math.abs(lift.floor() - call.floor()); }
}

class LeastTravel implements DispatchStrategy {             // follows the lift's direction first
    public int cost(Lift lift, HallCall call) {
        int f = lift.floor(), c = call.floor();
        if (lift.direction() == Direction.NONE) return Math.abs(c - f);
        boolean ahead = lift.direction() == Direction.UP ? c > f : c < f;
        if (ahead) return Math.abs(c - f);
```

```
        int end = lift.farthestStopAhead();
        return Math.abs(end - f) + Math.abs(end - c);
    }
}

// ---------------- listeners ----------------
class HallButtons implements LiftListener {                     // only used under the system's lock
    private final Map<HallCall, Lift> lit = new HashMap<>();

    Lift answering(HallCall call)       { return lit.get(call); }
    void light(HallCall call, Lift lift) { lit.put(call, lift); }

    public void onStop(Lift lift, int floor) {                  // it arrived: its buttons here go dark
        lit.entrySet().removeIf(e -> e.getKey().floor() == floor && e.getValue() == lift);
    }
}

class Display implements LiftListener {
    private final Clock clock;
    private final List<String> lines = new ArrayList<>();

    Display(Clock clock) { this.clock = clock; }

    public void onStop(Lift lift, int floor) {
        lines.add("t=" + clock.now() + ": " + lift.id + " stops at " + floor);
    }

    List<String> drain() {
        List<String> out = new ArrayList<>(lines);
        lines.clear();
        return out;
    }
}

enum CallStatus { ASSIGNED, ALREADY_ASSIGNED, ALREADY_HERE, WAITING, ALREADY_WAITING }

record CallResult(HallCall call, CallStatus status, Lift lift) {}

// ---------------- the front door: one lock around every operation ----------------
class ElevatorSystem {
    private static final Map<String, Supplier<DispatchStrategy>> STRATEGIES = Map.of(
            "NEAREST", NearestLift::new,
            "LEAST_TRAVEL", LeastTravel::new);

    private final int floors;
    private final Clock clock;
    private final HallButtons buttons = new HallButtons();
    private final List<Lift> lifts = new ArrayList<>();
    private final Deque<HallCall> waiting = new ArrayDeque<>();  // made while no lift was in service
    private DispatchStrategy strategy = new NearestLift();

    ElevatorSystem(int floors, int liftCount, Clock clock, LiftListener display) {
        this.floors = floors;
        this.clock = clock;
        List<LiftListener> listeners = List.of(buttons, display);
        for (int i = 1; i <= liftCount; i++) lifts.add(new Lift("L" + i, listeners));
    }

    synchronized CallResult call(int floor, Direction direction) {
        checkFloor(floor);
        boolean top = floor == floors - 1, ground = floor == 0;
        if ((direction == Direction.UP && top) || (direction == Direction.DOWN && ground))
            throw new LiftException("No " + direction + " button at floor " + floor);
        HallCall call = new HallCall(floor, direction);
        Lift answering = buttons.answering(call);               // check and assign: one step, one lock
        if (answering != null) return new CallResult(call, CallStatus.ALREADY_ASSIGNED, answering);
        if (waiting.contains(call)) return new CallResult(call, CallStatus.ALREADY_WAITING, null);
        return dispatch(call);
    }

    // Returns false if the lift's doors are already open at that floor.
    synchronized boolean go(String liftId, int floor) {
        Lift lift = lift(liftId);
        checkFloor(floor);
        if (lift.doorsOpenAt(floor)) return false;
        lift.addStop(floor);                                    // the lift's state may refuse
        return true;
    }

    synchronized void step(int ticks) {
        for (int i = 0; i < ticks; i++) {
            clock.tick();
            for (Lift l : lifts) l.tick();
        }
    }

    synchronized List<CallResult> maintenance(String liftId, boolean on) {
        Lift lift = lift(liftId);
        if (on) {
            lift.maintenanceOn();
            return List.of();
        }
        lift.maintenanceOff();
        List<HallCall> queued = new ArrayList<>(waiting);       // a lift is back: serve the queue
        waiting.clear();
        List<CallResult> results = new ArrayList<>();
        for (HallCall call : queued) results.add(dispatch(call));
        return results;
    }

    synchronized void setStrategy(String name) {
```

```
        Supplier<DispatchStrategy> maker = STRATEGIES.get(name);
        if (maker == null) throw new LiftException("Unknown strategy " + name);
        strategy = maker.get();
    }

    synchronized List<String> status() {
        List<String> out = new ArrayList<>();
        out.add("t=" + clock.now());
        for (Lift l : lifts) out.add(l.status());
        if (!waiting.isEmpty()) out.add("Waiting: " + waiting);
        return out;
    }

    private CallResult dispatch(HallCall call) {
        for (Lift l : lifts)
            if (l.doorsOpenAt(call.floor())) return new CallResult(call, CallStatus.ALREADY_HERE, l);
        List<Lift> available = lifts.stream().filter(Lift::inService).toList();
        if (available.isEmpty()) {
            waiting.add(call);
            return new CallResult(call, CallStatus.WAITING, null);
        }
        Lift chosen = strategy.choose(call, available);
        chosen.addStop(call.floor());
        buttons.light(call, chosen);
        return new CallResult(call, CallStatus.ASSIGNED, chosen);
    }

    private Lift lift(String id) {
        return lifts.stream().filter(l -> l.id.equals(id)).findFirst()
                .orElseThrow(() -> new LiftException("Unknown lift " + id));
    }

    private void checkFloor(int floor) {
        if (floor < 0 || floor >= floors) throw new LiftException("Floor " + floor + " does not exist");
    }
}

public class Main {
    public static void main(String[] args) throws InterruptedException {
        ElevatorSystem system = null;
        Display display = null;
        Scanner in = new Scanner(System.in);
        while (in.hasNextLine()) {
            String line = in.nextLine().trim();
            if (line.isEmpty()) continue;
            String[] p = line.split("\\s+");
            try {
                if (system == null && !p[0].equals("BUILDING"))
                    throw new LiftException("No building set up");
                switch (p[0]) {
                    case "BUILDING" -> {
                        int floors = Integer.parseInt(p[1]), count = Integer.parseInt(p[2]);
                        if (floors < 2 || count < 1)
                            throw new LiftException("A building needs at least 2 floors and 1 lift");
                        Clock clock = new Clock();
                        display = new Display(clock);
                        system = new ElevatorSystem(floors, count, clock, display);
                        System.out.println("Building ready: floors 0-" + (floors - 1) + ", "
                                + count + (count == 1 ? " lift" : " lifts"));
                    }
                    case "CALL" -> System.out.println(
                            describe(system.call(Integer.parseInt(p[1]), Direction.parse(p[2]))));
                    case "GO" -> {
                        int floor = Integer.parseInt(p[2]);
                        System.out.println(system.go(p[1], floor) ? p[1] + " will stop at " + floor
                                                                  : p[1] + " is already at " + floor);
                    }
                    case "STEP" -> system.step(Integer.parseInt(p[1]));
                    case "STATUS" -> system.status().forEach(System.out::println);
                    case "MAINT" -> {
                        boolean on = p[2].equals("ON");
                        List<CallResult> assigned = system.maintenance(p[1], on);
                        System.out.println(p[1] + (on ? " is out of service for maintenance"
                                                      : " is back in service"));
                        assigned.forEach(r -> System.out.println(describe(r)));
                    }
                    case "STRATEGY" -> {
                        system.setStrategy(p[1]);
                        System.out.println("Strategy: " + p[1]);
                    }
                    case "RUSH" -> System.out.println(rush(system, Integer.parseInt(p[1]),
                            Integer.parseInt(p[2]), Direction.parse(p[3])));
                    default -> System.out.println("Unknown command: " + p[0]);
                }
            } catch (LiftException e) {
                System.out.println(e.getMessage());
            }
            if (display != null) display.drain().forEach(System.out::println);
        }
    }

    static String describe(CallResult r) {
        return "Call " + r.call() + ": " + switch (r.status()) {
            case ASSIGNED -> "assigned to " + r.lift().id;
            case ALREADY_ASSIGNED -> "already assigned to " + r.lift().id;
            case ALREADY_HERE -> r.lift().id + " is already here";
            case WAITING -> "no lift in service, waiting";
            case ALREADY_WAITING -> "already waiting";
        };
    }
```

```
    // n people press the same hall button at the same moment, from n threads
    static String rush(ElevatorSystem system, int n, int floor, Direction direction)
            throws InterruptedException {
        ExecutorService pool = Executors.newFixedThreadPool(n);
        CountDownLatch startGun = new CountDownLatch(1);
        try {
            List<Future<CallResult>> results = new ArrayList<>();
            for (int i = 0; i < n; i++)
                results.add(pool.submit(() -> {
                    startGun.await();
                    return system.call(floor, direction);
                }));
            startGun.countDown();
            int assigned = 0, duplicates = 0;
            for (Future<CallResult> f : results) {
                try {
                    if (f.get().status() == CallStatus.ASSIGNED) assigned++;
                    else duplicates++;
                } catch (ExecutionException e) {
                    if (e.getCause() instanceof LiftException le) throw le;
                    throw new IllegalStateException(e.getCause());
                }
            }
            return "Rush: " + assigned + " assigned, " + duplicates + " already assigned";
        } finally {
            pool.shutdown();
        }
    }
}
```

How to read it: the four state classes follow the R6 table. The `TreeSet` does the "keep going" work: `higher` and `lower` answer "is there a stop ahead?", `ceiling` and `floor` find the nearest stop either way. `ElevatorSystem.call()` is the indivisible step of answer 4. `dispatch()` is shared by new calls and by the waiting queue, so both follow the same rules, including "already here".

Part C — model answers

1. FIRE mode. A new state, `FireState`: its tick moves toward floor 0 without stopping, opens the doors there, and then does nothing; it refuses stops and maintenance. The system gets a `fireAlarm()`: for every lift, clear its stops, point it toward 0 and set it to FIRE; turn off every lit button, empty the waiting queue, and refuse calls until a reset. The existing states don't change (NF3). One question to raise: what about a lift that's in maintenance, possibly with a technician on top of it? If entering FIRE depends on the current state, make `fire()` an operation of every state, and let the compiler list the states that need an answer.

2. Respect the call's direction. Keep hall stops with their direction: `upStops` and `downStops` (two `TreeSet`s), plus `carStops`. A lift going up stops for car stops and UP calls; it serves DOWN calls on its way down, or at the top of its trip where it turns around. `HallButtons` already keys on floor and direction, so it turns off only the button that was served. LEAST_TRAVEL can use the direction too: "ahead and the same way" is cheap, "ahead but the other way" means after the turnaround. What changes: the lift's stop sets and its "is this floor a stop?" test. The strategies stay interchangeable, and the states barely move.

3. L2 breaks down. A new state, OUT_OF_ORDER, entered from any state (much like maintenance, but forced). The system finds every lit button assigned to L2 and dispatches it again: to another lift, or into the waiting queue if none is in service. That's easy because `HallButtons` knows which lift answers each button. L2's car calls are dropped; the people inside need the alarm and a technician, which is outside the software. Announce the fault on the display.

4. A lock per lift. The dispatcher holds the system lock and then needs a lift's lock to add a stop. A lift's thread holds its own lock during its tick, and then needs the system lock when `onStop` reaches `HallButtons`. Two locks taken in opposite orders: that's a **deadlock**. The fixes: (a) keep one lock, which is fine at this scale; (b) one fixed order, system lock then lift lock, with the lift notifying its listeners only *after* releasing its own lock (announce outside the lock); (c) message passing: each lift thread owns its data and reads commands from a `BlockingQueue`, and only the dispatcher touches the lit buttons. Option (c) avoids shared data altogether.

5. "An enum and a switch would do." "You'd be right if only `tick()` depended on the state. Here all four operations do, so the same switch would repeat in four methods. FIRE and OUT_OF_ORDER are already on the way, and each would mean editing all four. With State, each mode is one class with all its rules in one place, and FIRE doesn't touch the others. For a lift with two modes and one operation, I'd use the enum."

Common mistakes

- Mixing dispatch and movement in one class, so the lift decides which hall calls it takes.
- Checking the lit button and lighting it in two separate locked sections.
- Turning around while there are still stops ahead, so the lift ping-pongs.
- Announcing a floor the lift only passed, or forgetting to remove the stop when it arrives.
- Never turning a hall button off, so every later press says "already assigned".
- Losing calls made while every lift is in maintenance, instead of queueing them.
- Using `Thread.sleep` for time: the tests become slow and flaky. A tick counter gives the same output every run.
- Exact text: `t=7: L1 stops at 6`; STATUS shows stops only when there are some.

Score yourself (1 point each)

☐	The lift's states are classes holding no data; the lift holds floor, direction and stops
☐	Your tick follows R6, including "keep going while there are stops ahead"
☐	Dispatch is a Strategy chosen by name, and it never moves a lift
☐	LEAST_TRAVEL follows the formula in R7, with ties to the lower number
☐	The lift announces stops and knows nothing about buttons or displays
☐	Checking, choosing, adding the stop and lighting the button happen under one lock
☐	Calls wait in a queue while no lift is in service, and are served in order
☐	Your output matches exactly, RUSH included
☐	Part C 4: you named the deadlock and a fix
☐	Part C 5: you opened with a concession

Back pages

Warm-up answers

W1 — Vending Machine. The state table:

	select	insert	cancel	service on	service off	restock
IDLE	check slot and stock → SELECTED	reject a bad coin, else "select first"	"Nothing to cancel"	→ SERVICE	"not in service"	"needs service mode"
SELECTED	"finish or cancel first"	check the coin, add it, maybe complete	refund → IDLE	"during a purchase"	"not in service"	"needs service mode"
SERVICE	"in service mode"	"in service mode"	"in service mode"	"already"	→ IDLE	allowed

The final coin, in order: (1) copy the coin stock; (2) add the inserted coins to the *copy*; (3) try to make change from the copy; if that fails, refund, go back to IDLE, then throw; (4) if it works, commit: add the inserted coins to the real stock, pay the change, reduce the quantity, go back to IDLE.

W2 — Parking Lot. One indivisible step: "is this vehicle already parked?" + choose a spot + occupy it + create the ticket. Outside the lock: telling the listeners (the display), because a slow or crashing display must not hold up every gate. The designs: the policy by name is **Strategy + Factory**; the weekend surcharge is a **Decorator** around the hourly fee; the display board is an **Observer**; the vehicle rates are an **enum with fields**, with no pattern.

#	W3 answer	Why
1	**Strategy**	a different formula per kind, changing yearly. (If they differed only in a rate, an enum with fields.)
2	**Chain of Responsibility**	up to X this one decides; otherwise pass it on.
3	**State**	several operations, each depending on the status.
4	**Nothing**: a scheduled job	"hasn't happened in N days" isn't an event. Run a daily job.
5	**Adapter**	a vendor's shape that we can't edit, behind our own interface.
6	**Decorator**	extras that stack in any combination.
7	**Observer**	reactions to one event, and more to come. (Starting the pump is arguably a required step: if it fails, someone must know.)
8	**Composite**	a tree where a whole and its parts answer the same questions.
9	**Proxy** (a guard)	same interface as the archive, with a check and a log in front.
10	**Builder**	required plus many optional parts, never changing afterwards.
11	**Nothing**: a record	two fields that never change.
12	**Facade**	several steps in order, behind one call, used from two places.

Both designs right and 10 or more of the 12: on track. If W1 or W2 was shaky, re-do that problem's Part A from PDF 1 before you start Problem 2 here.

Mistake log

One row per mistake, in your own words. Before starting PDF 3, read this page and PDF 1's log once: these are the mistakes most likely to come back.

Date	Problem	What I did	What's right	Rule to remember

Re-do schedule

Problem	Attempted (time taken)	Day 3: Part A from memory (15 min)	Day 10: Part A + the core class	Day 30: the whole problem, timed	Score /10
Deck of Cards					
ATM					
Splitwise					
Elevator System					

The core class to rebuild on day 10: `Hand` with its comparator · `Atm.dispense()` with the cassette chain · `SplitService.addExpense()` with `Shares.byWeight` · `Lift` with its four states. PDF 1's day-30 re-dos are probably due around now too.

> **Next: PDF 3.** LRU Cache, Rate Limiter, Digital Wallet and BookMyShow. Its warm-up will ask you to re-design the ATM and the Elevator System from memory, so the day-10 re-dos above are worth doing before you open it.